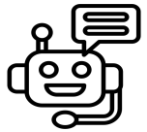
Advanced Topics of AI Agents

Intelligent Chatbot/HITL

Types of AI-Powered Solutions

- Workflow 
  - Automated **process pipeline**

- Intelligent Chatbot 
  - The system could perform **iterative queries and analysis** based on the end user queries

- Agentic Assistant 
  - It will understand, plan and execute the tasks **autonomously**

Intelligent Chatbot

The query will be broken down
Into two sub-queries by LLM.

Q: Do you have any [product] with features A and B which is less than 5000 ?

Customer query AI chatbot



User Interface







Final Response to end user

Text-to-SQL

Sub-query 1: Any product with price less than 5000 ?
⇒ SQL statement
⇒ Execute the SQL statement and retrieve the answer

Relational Database
(Structured data)



Document Database
(Unstructured data)



LLM

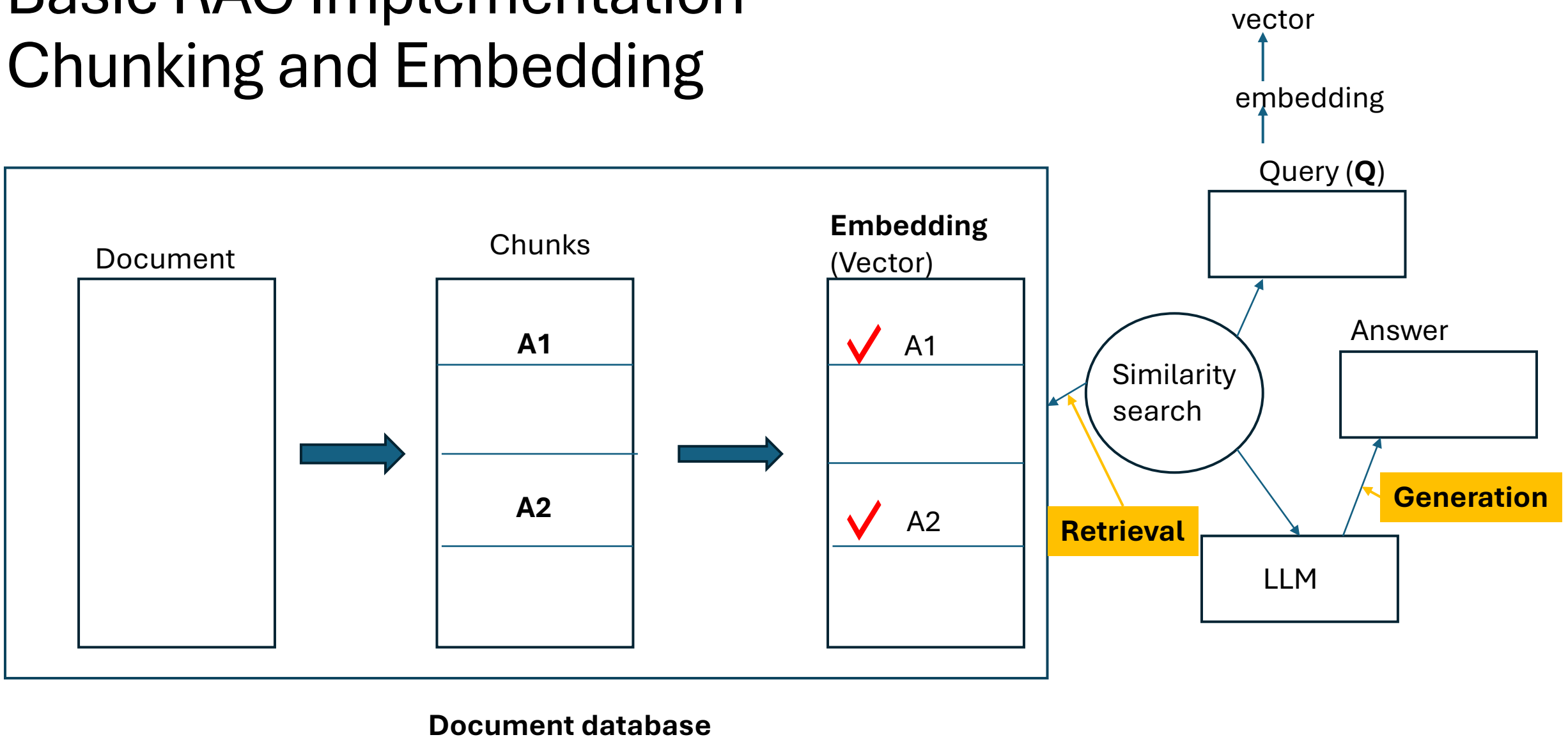
Sub-query 2:
Do they support features A & B ?
->RAG mechanism

RAG

AI Solution Framework

Retrieval-Augmented Generation (RAG) with Vector Database

Basic RAG Implementation
Chunking and Embedding



Embedding and Similarity Comparison

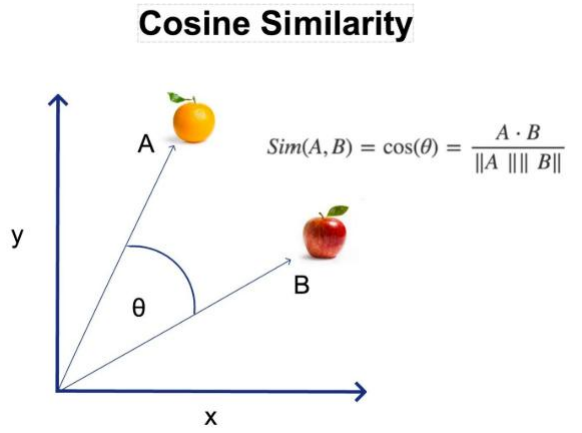
Apple

Orange

vector

```
-0.01871234,  0.03124567, -0.00782345,  0.04415678, -0.02133456,
 0.04151234, -0.00391234,  0.06342310, -0.00123456,  0.08567890,
-0.05123456,  0.01782345, -0.07234567,  0.01567890, -0.03678901,
 0.06123456, -0.01894567,  0.04678901, -0.08345678,  0.00912345,
 0.03456789, -0.01098765,  0.06901234, -0.02789012,  0.01543210,
 0.05234567, -0.02210987,  0.04198765, -0.07890123,  0.00678901,
... (continues exactly 1536 values) ...
-0.03912345,  0.02678901, -0.05432109,  0.03210987, -0.01456789,
-0.04210987,  0.02345678, -0.05789012,  0.02987654, -0.01345678
```

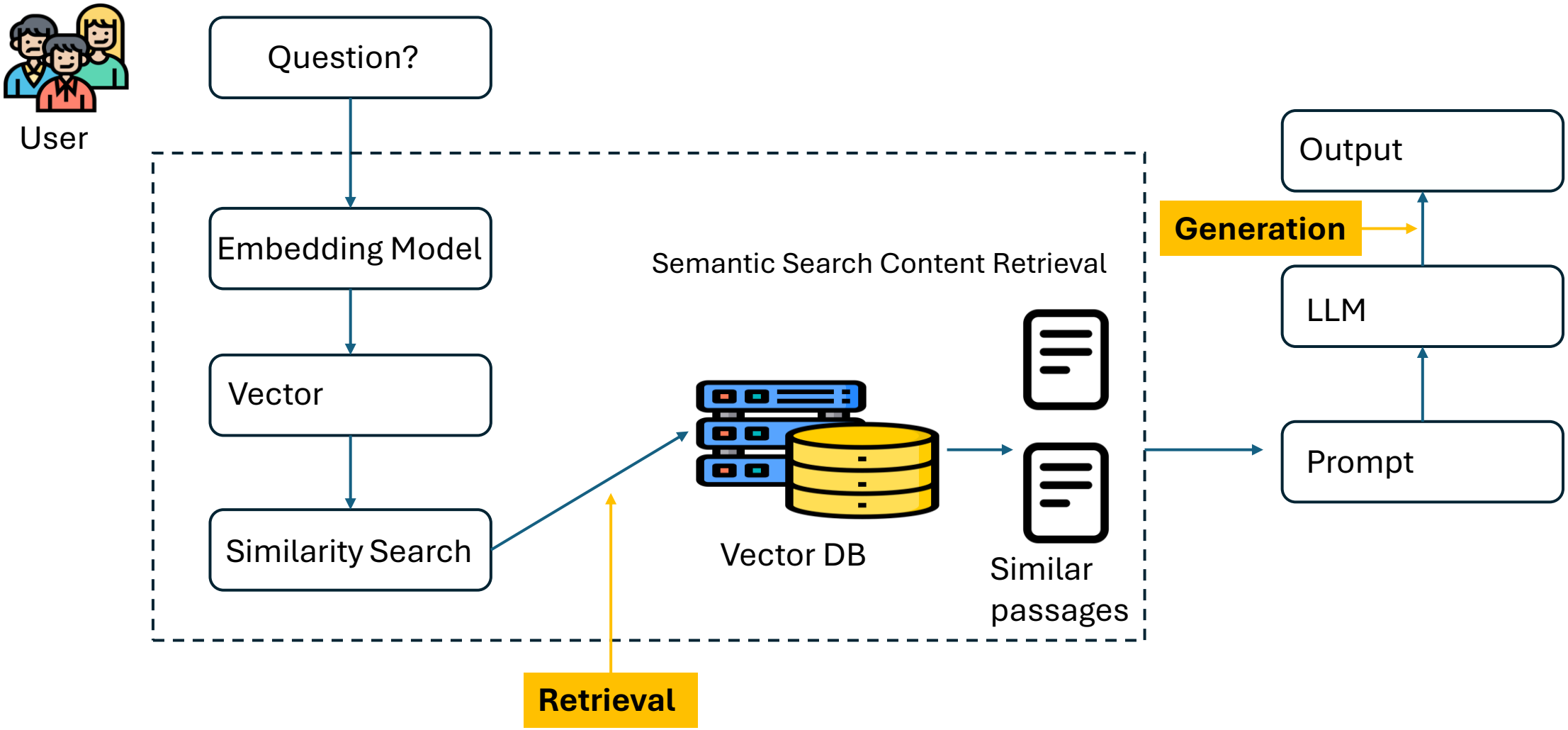
```
-0.01984567,  0.03256789, -0.00691234,  0.04578901, -0.02012345,
 0.04298765, -0.00278901,  0.06512345, -0.00098765,  0.08734567,
-0.04987654,  0.01898765, -0.07012345,  0.01678901, -0.03545678,
 0.06345678, -0.01765432,  0.04812345, -0.08123456,  0.01034567,
 0.03389012, -0.01123456,  0.06890123, -0.02678901,  0.01654321,
 0.05123456, -0.02198765,  0.04321098, -0.07654321,  0.00789012,
... (exactly 1536 values total) ...
-0.04098765,  0.02567890, -0.05678901,  0.03123456, -0.01234567,
-0.04123456,  0.02456789, -0.05567890,  0.03098765, -0.01189012
```



Similarity comparison
0.842

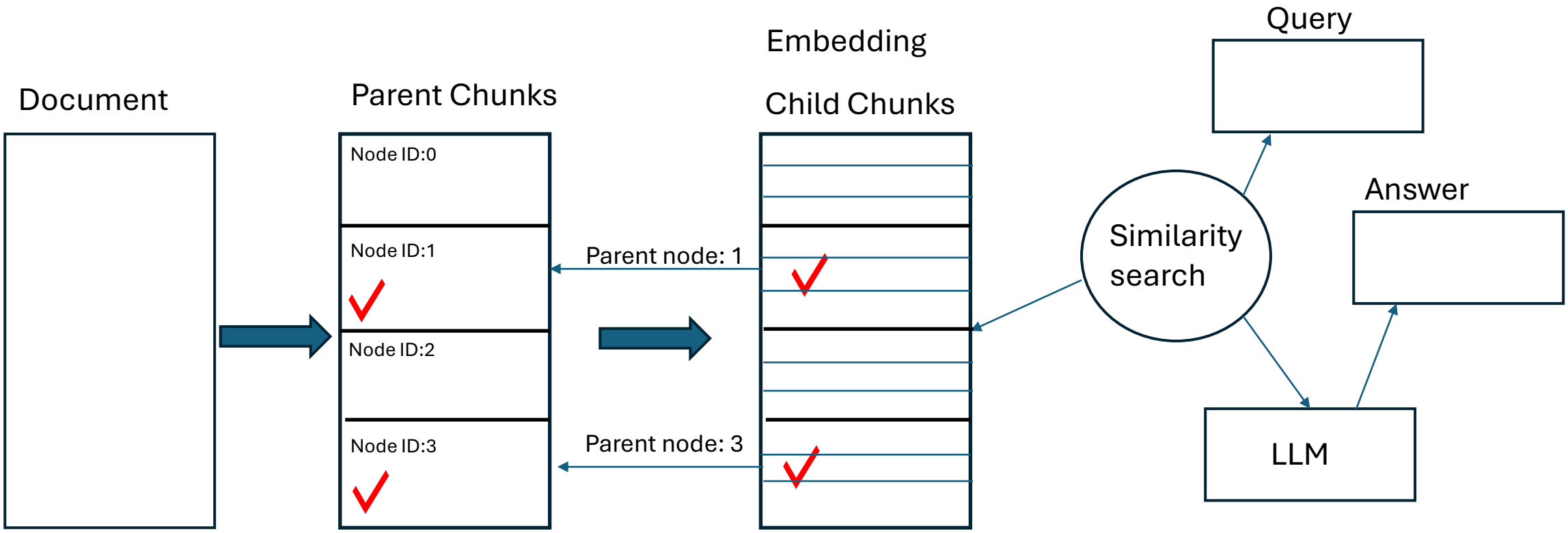
Similarity from 0 to 1

Basic RAG Implementation



Retrieval Improvement Techniques

Parent Document Retriever implementation

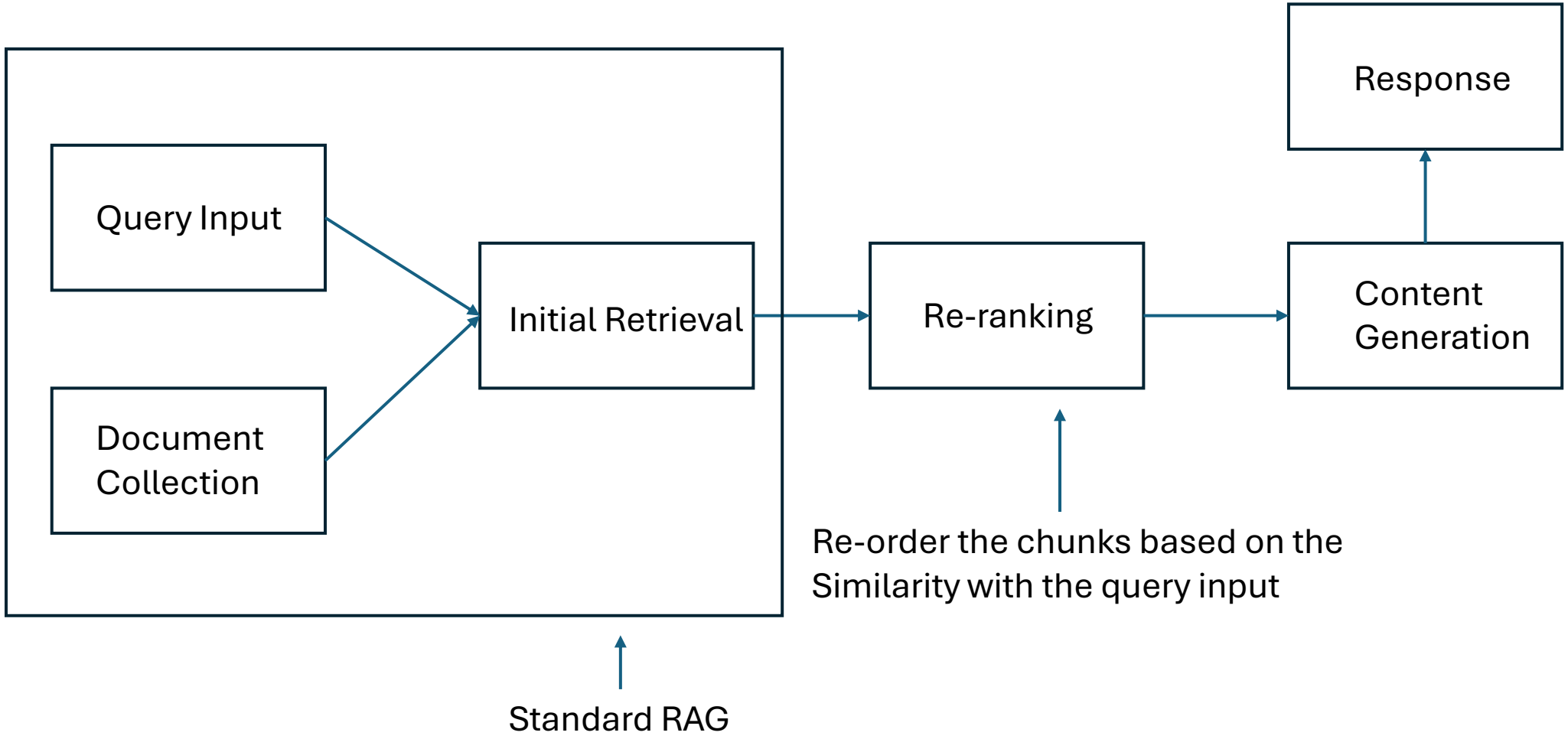


Benefits:

-Easy to search and identify the child chunks with high similarity to the query

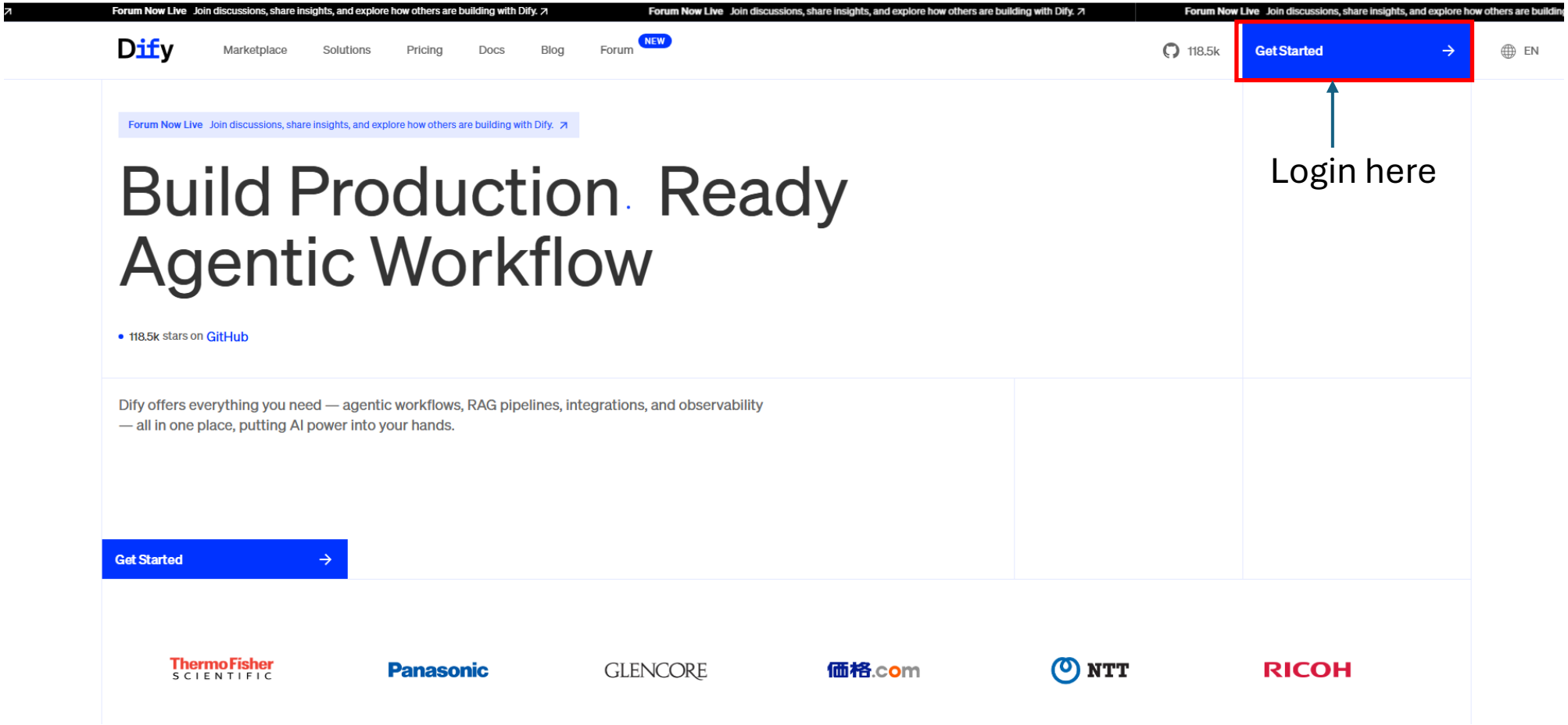
-Parent chunks could then be linked and identified from the child chunks and all of useful information will be sent to LLM for the generation of the reply

Re-ranking Techniques

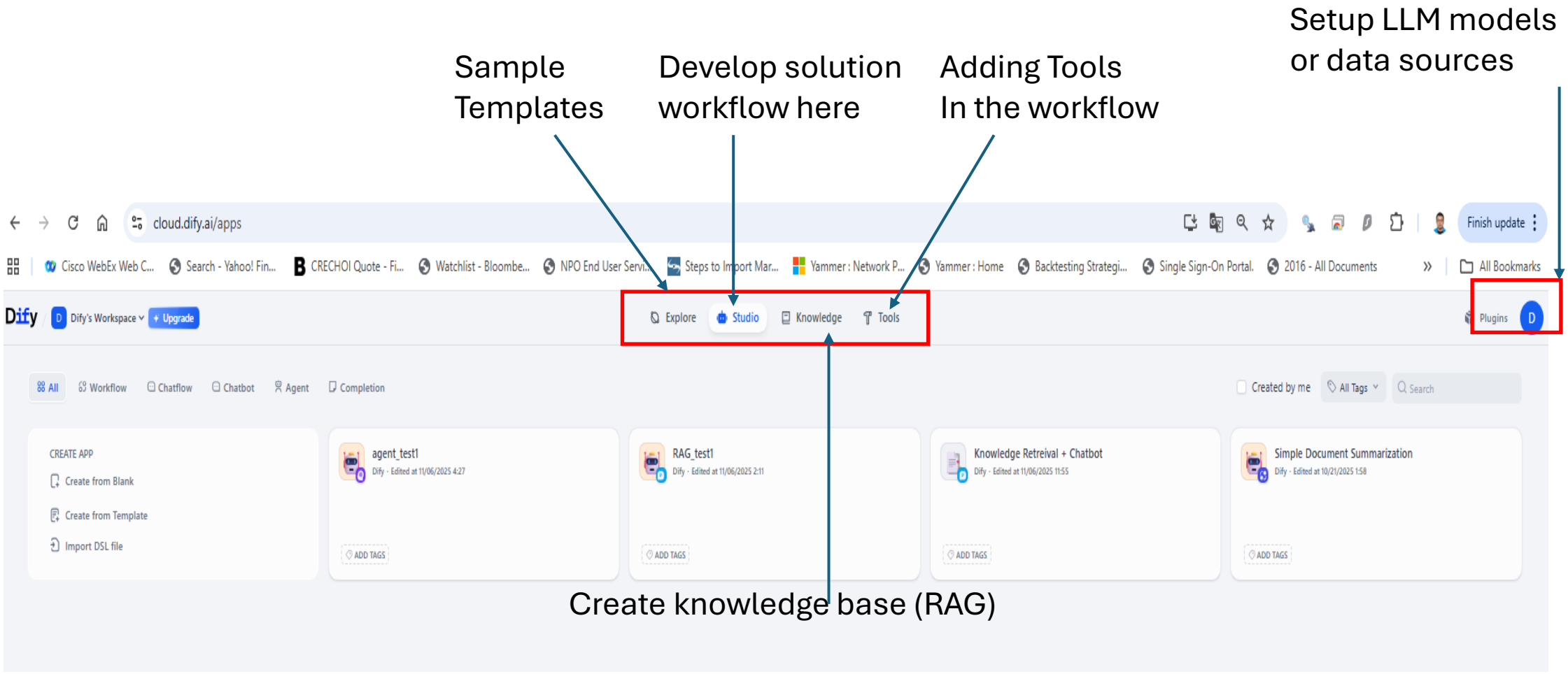


Setup of Dify

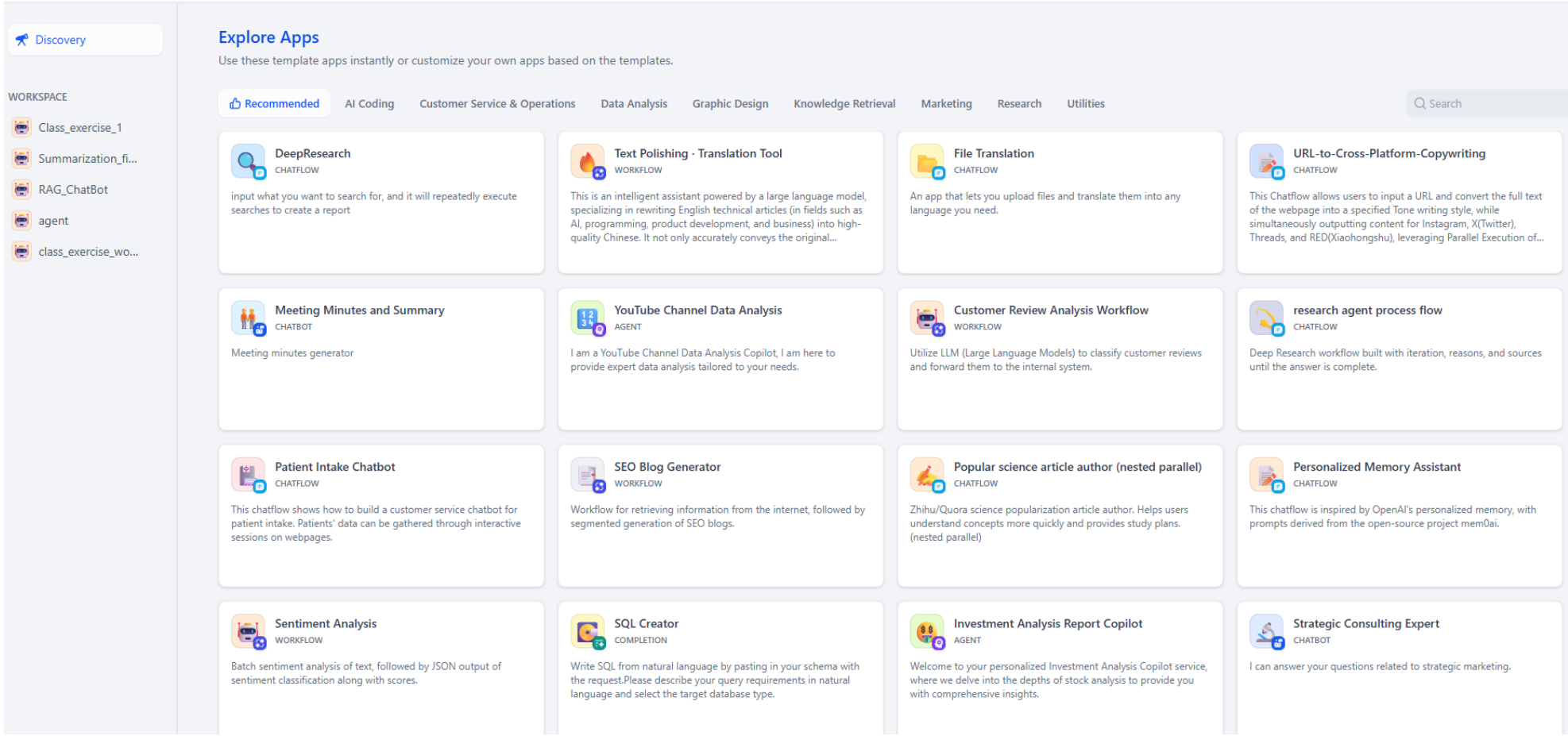
Overall setup



Overall setup



Templates from Explore



Dify

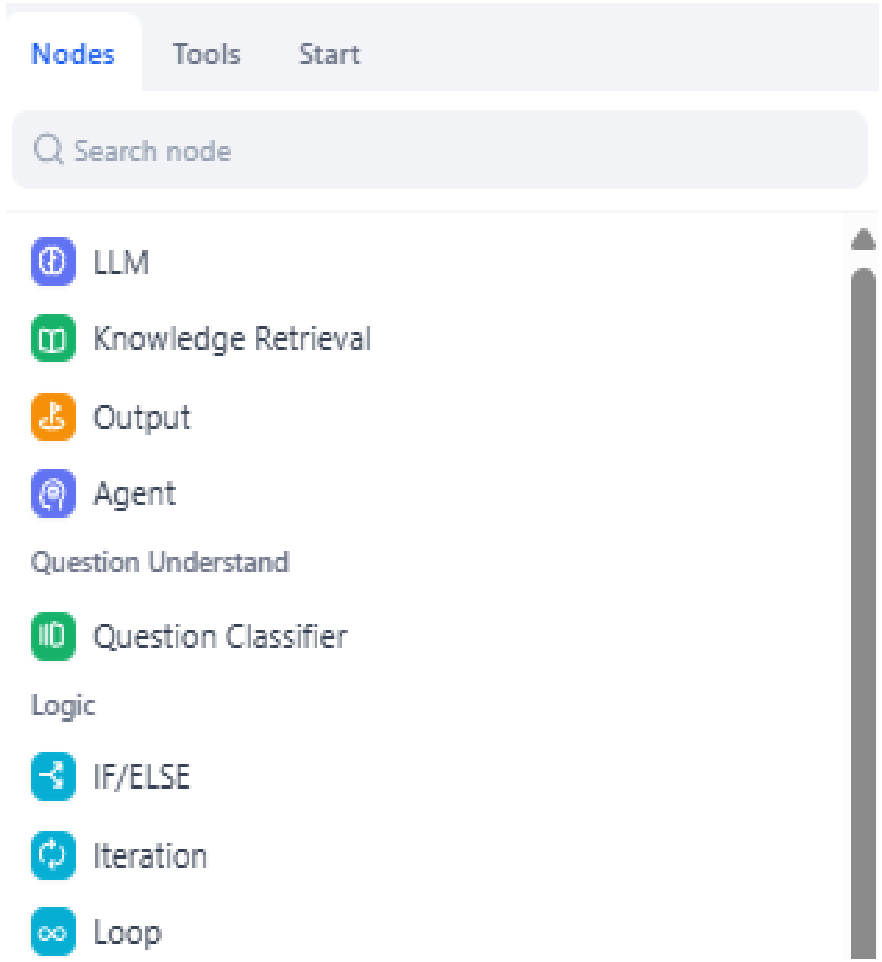
Overall Development Process

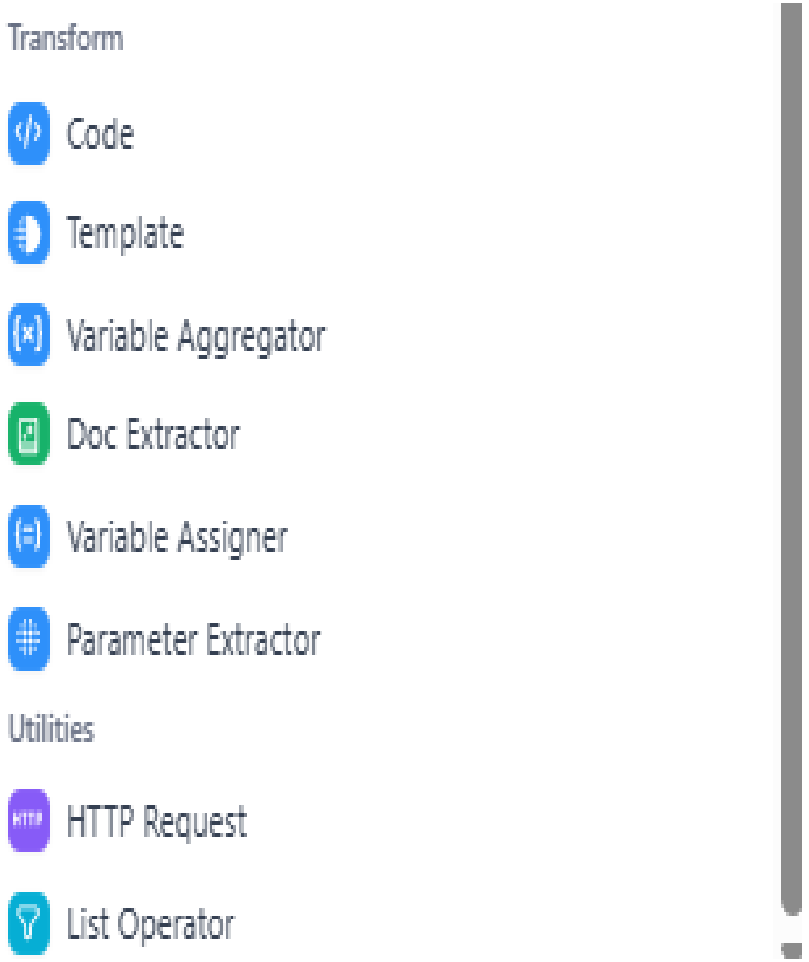
Development Workflow

Item	Task	Details	Comment
1	Define the scope	Decide: internal knowledge base? customer support?	
2	Document Understanding & Systematic Structuring	Metadata schema (source, version, owner, last-updated, doc_type, tags, access_level) Hierarchical structure (document → section → subsection → chunk) Cleaning & normalization pipeline (remove headers/footers, OCR fixes, table extraction)	80% of RAG performance lives here
3	Golden Dataset Creation	Collect 150–300 real user questions Expert-written ground-truth answers + exact citations	This becomes your automated test suite forever
4	Chunking Strategy & Metadata Enrichment	Test 3–4 strategies (fixed 512 tokens, semantic, hierarchical, agentic chunking) and measure recall on your golden set	Wrong chunk size = catastrophic recall drop
5	Embedding Model & Vector Store Selection	Choose vector DB that supports hybrid search + metadata filtering	
6	Advanced Retrieval Pipeline	Deploy advanced data processing pipelines -hybrid searching/reranking/graphRAG etc Deploy automated documents update pipeline	
7	Generation & Guardrails	Prompt engineering + structured output• Citation enforcement (LLM must return source IDs) Refusal / uncertainty detection• PII / compliance guardrails	
8	Automated Evaluation Framework	Use your golden dataset to run daily/nightly regression tests: Context Recall & Precision Faithfulness (LLM-as-judge) Answer Correctness Citation accuracy Latency + cost	You cannot improve what you cannot measure

Types of Nodes

Types of nodes

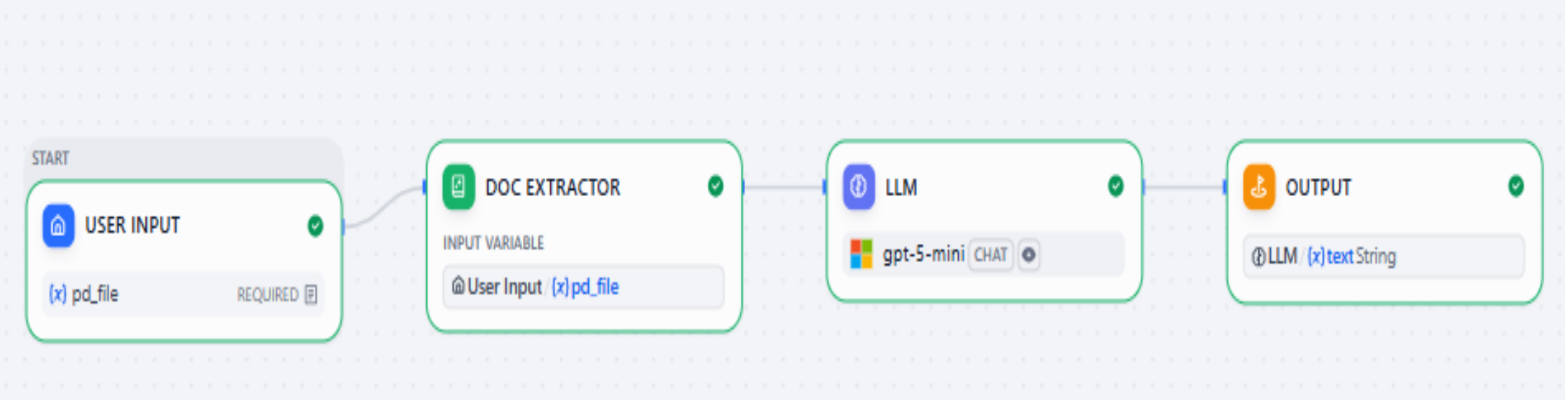




Core AI and Output Nodes

Node Name	Description	Configuration Basics	Use Cases & Examples
LLM	Invokes a Large Language Model (e.g., GPT-4, Claude, or local models) to generate text, reason, or extract info based on a prompt. Supports system/user/assistant roles and tools.	- Model selection (from connected providers). - Prompt template (with variables). - Parameters: Temperature (0-1 for creativity), max tokens. - Output: Text or structured JSON.	- Content generation: Prompt an LLM to summarize articles. - Q&A: Feed user query + context for natural responses. - Example: In a chatbot, connect input query to LLM prompt: "Answer: {{query}} using knowledge: {{retrieved_docs}}".
Knowledge Retrieval	Searches and retrieves relevant documents from a connected Knowledge Base (vector DB like Weaviate or Milvus) using semantic similarity.	- Select Knowledge Base. - Retrieval params: Top-K results, score threshold. - Query variable input. - Output: List of doc chunks with metadata (score, content).	- RAG (Retrieval-Augmented Generation): Pull context for LLM to ground responses. - Search engine: Query "latest AI news" to fetch and rank docs. - Example: User asks about "Dify features" → Retrieve top 5 docs → Pass to LLM for explanation.
Output	Defines the final response or endpoint output, aggregating variables into a structured format (e.g., JSON, text). Multiple outputs possible per workflow.	- Output type: Text, JSON, File. - Variable mapping (e.g., bind LLM output). - Streaming support for real-time responses.	- API responses: Return formatted JSON like {"answer": "{{llm_output}}"}. - Chat apps: Stream LLM generations to users. - Example: End of workflow: Output = "Summary: {{llm_result}}
Agent	An autonomous AI agent that chains LLM calls with tools, memory, and planning for complex tasks. Supports ReAct or custom reasoning loops.	- LLM model and prompt for planning. - Attach tools (e.g., HTTP, Code). - Memory: Short-term (conversation) or long-term (vector store). - Max iterations to prevent loops.	- Multi-step automation: Agent books flights by querying APIs, reasoning over results. - Research assistant: Searches web, summarizes, verifies facts. - Example: "Plan a trip to Tokyo" → Agent uses tools to fetch prices, itineraries, then outputs plan.

Sample Workflow
Document Summarization Engine



It defines the path of the files being used

It extract the content of the files as JSON file

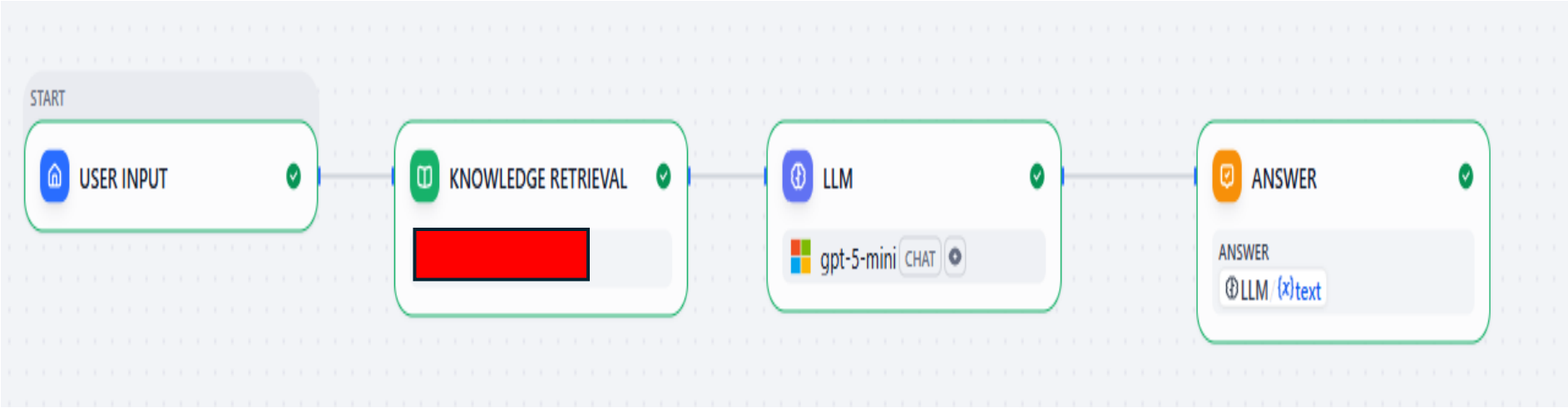
Perform content Summarization based on the prompts

It shows the end result

Class Exercise One (Part A)

RAG with Vector DB

Intelligent Chatbot
Sample RAG implementation



It triggers the flow and end user could put the query here

It retrieves relevant chunks from the knowledge base

It uses the chunks and the query and compose the response

It shows the response here

Setup of workflow (RAG) in Dify

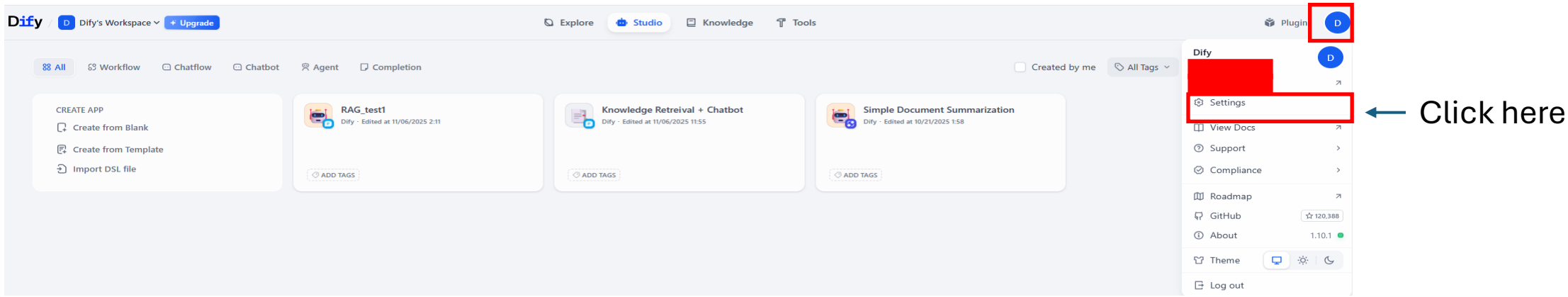
- Setup LLM
- Setup Embedding model
- Setup vector DB (Knowledge base)
- Setup workflow

LLM Setup

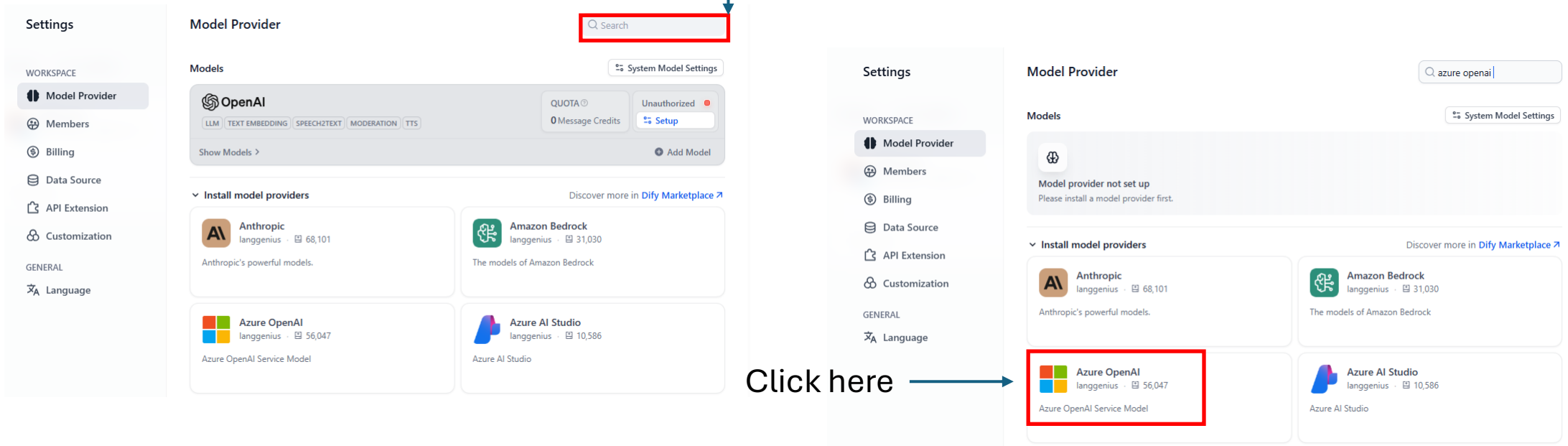
Types of LLM

What your AI agent needs to do	Best LLM type to pick	Top examples (2025 updates)	Main reason
Be super smart & handle almost any task	Flagship all-rounders	Claude 4.1 Opus, Grok 4, GPT-4o, Gemini 2.5 Pro	Highest intelligence & versatility; excels in multi-step agent workflows
Run fast, cheap, or in huge numbers	Medium open models	Llama 4 70B, Qwen3 72B	Best speed/price ratio (~$0.20/M tokens); MoE for efficient scaling
Work on phone or stay 100% private/offline	Tiny on-device models	Llama 3.2 3B, Phi-3.5 mini, Gemma 3 2B	Runs locally on <4GB RAM; no cloud dependency for privacy
Read entire books, huge codebases, long videos	Long-context beasts	Gemini 2.5 Pro (2M+ tokens), Claude 4 (1M beta)	Remembers massive inputs; 80%+ accuracy on long-doc benchmarks
See and understand images, screenshots, charts	Vision models	GPT-4o, Gemini 2.5, Qwen2.5-VL	Native multimodal (text+image/video); top on VQA/OCR tasks

Model Setting-1/2



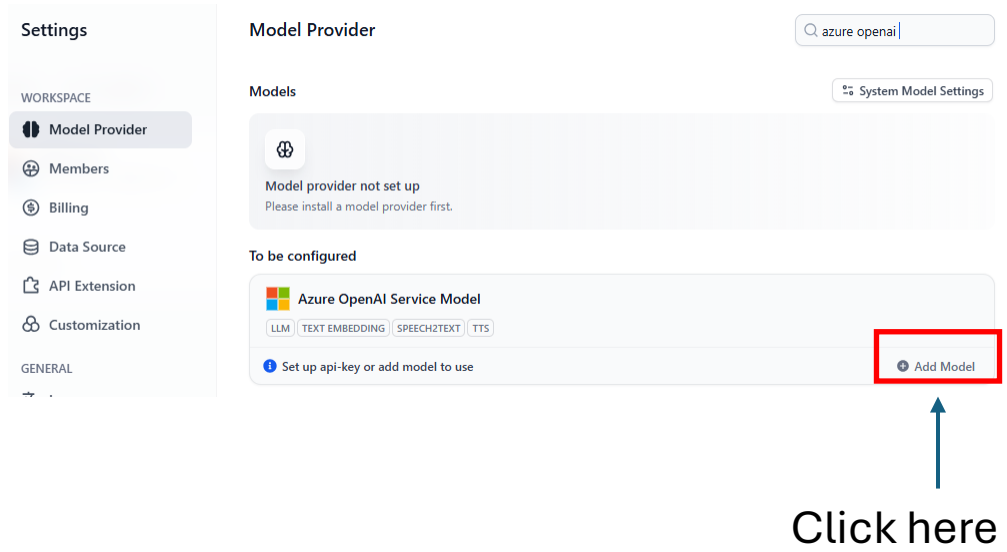
Search Azure OpenAI model here



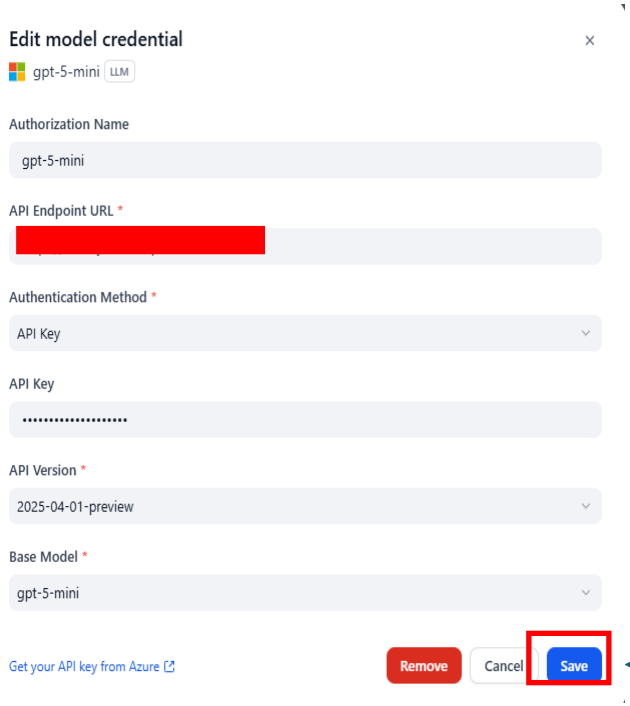
Model Setting-2/2

Refer to separate file for the detailed parameter setup

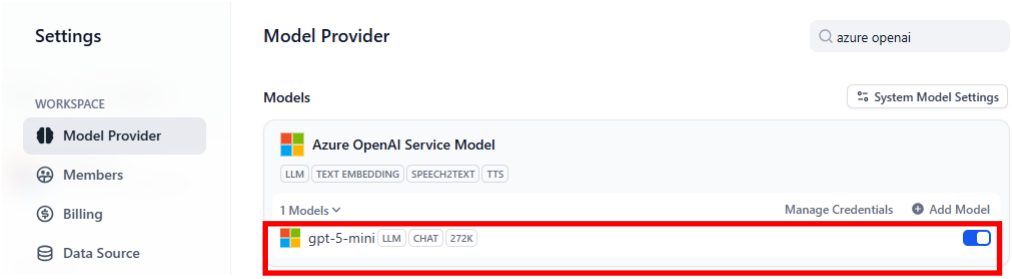
Remark: Use the parameters provided in the class



Click here

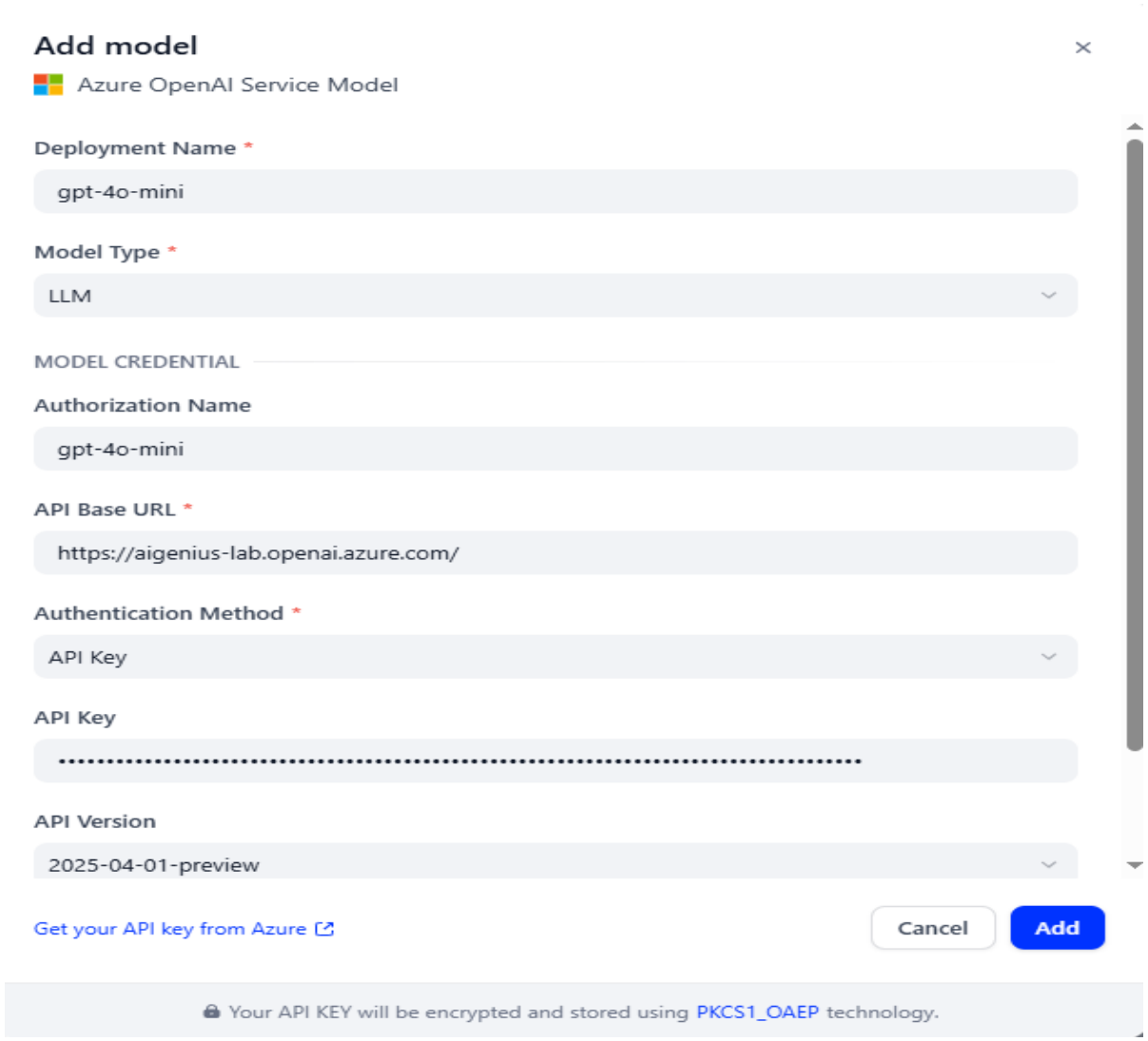


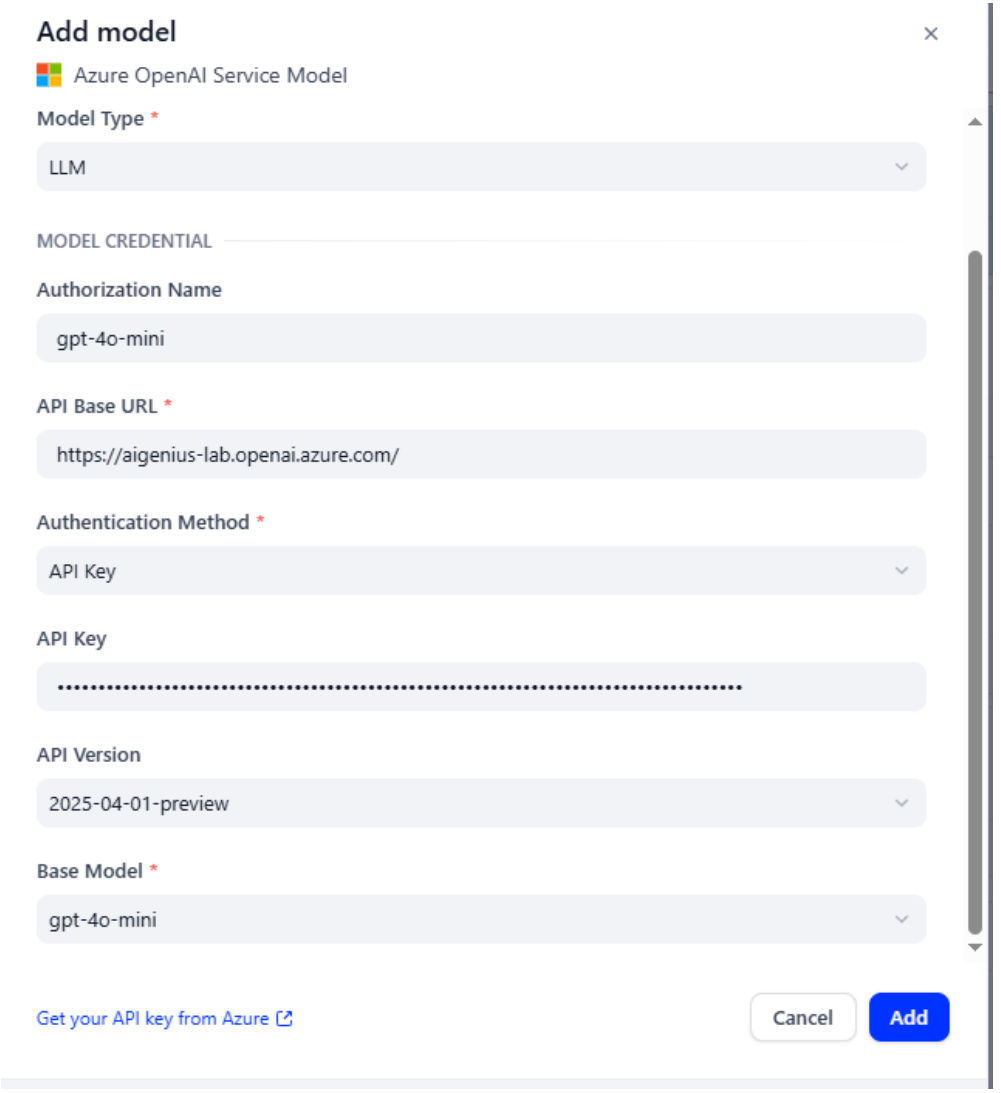
Click here



Successful installed

LLM setup

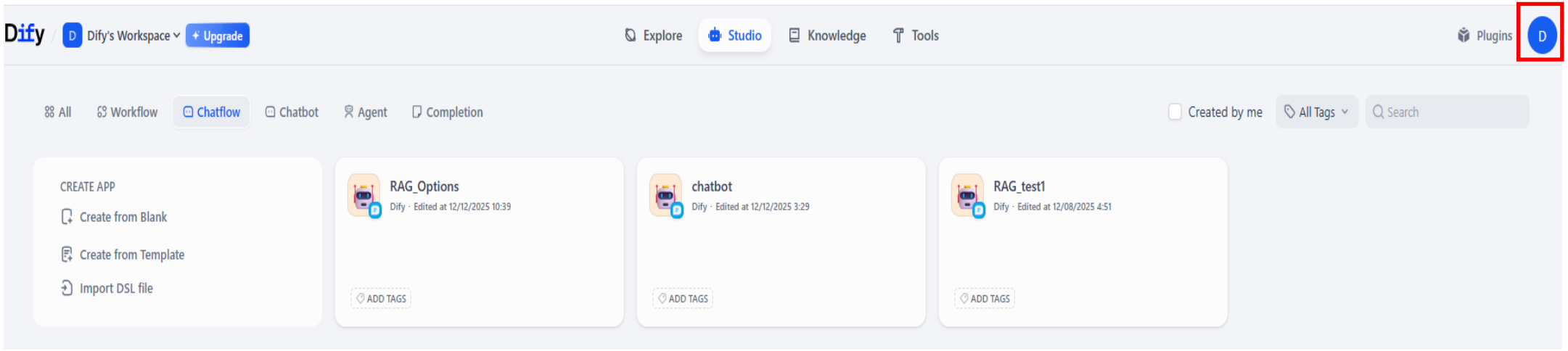

Add model
Azure OpenAI Service Model
Deployment Name *
gpt-4o-mini
Model Type *
LLM
MODEL CREDENTIAL
Authorization Name
gpt-4o-mini
API Base URL *
https://aigenius-lab.openai.azure.com/
Authentication Method *
API Key
API Key
API Version
2025-04-01-preview
Get your API key from Azure
Cancel
Add
Your API KEY will be encrypted and stored using PKCS1_OAEP technology.


Add model
Azure OpenAI Service Model
Model Type *
LLM
MODEL CREDENTIAL
Authorization Name
gpt-4o-mini
API Base URL *
https://aigenius-lab.openai.azure.com/
Authentication Method *
API Key
API Key
API Version
2025-04-01-preview
Base Model *
gpt-4o-mini
Get your API key from Azure
Cancel
Add

Embedding Model Setup

Embedding Model Setup-1/2

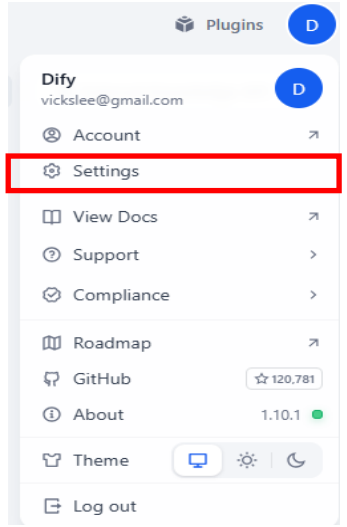
Setting of embedding model



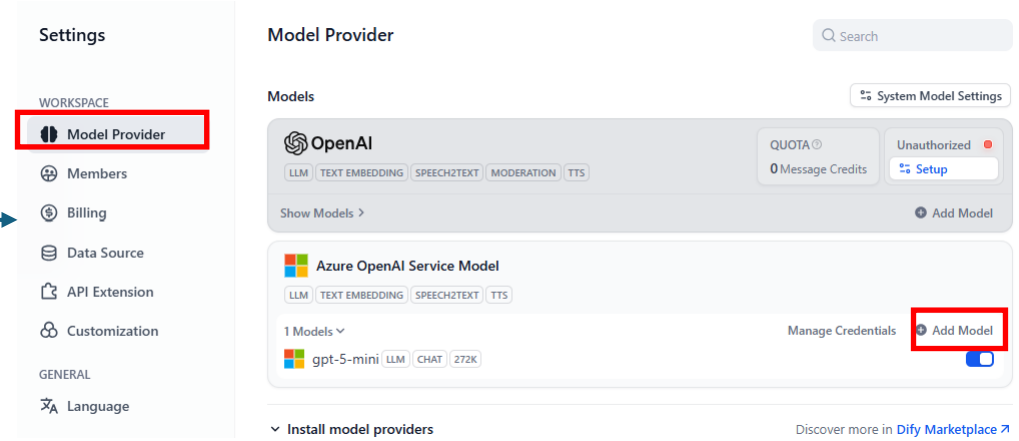
Embedding Model Setup-2/2

Remark: Use the parameters provided in the class

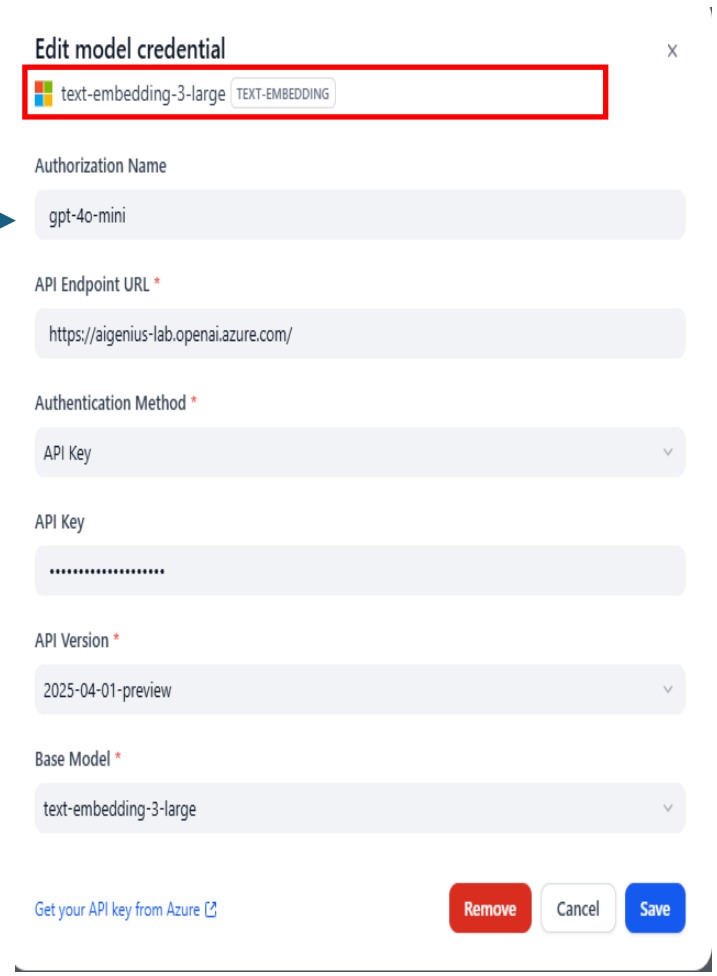
1.



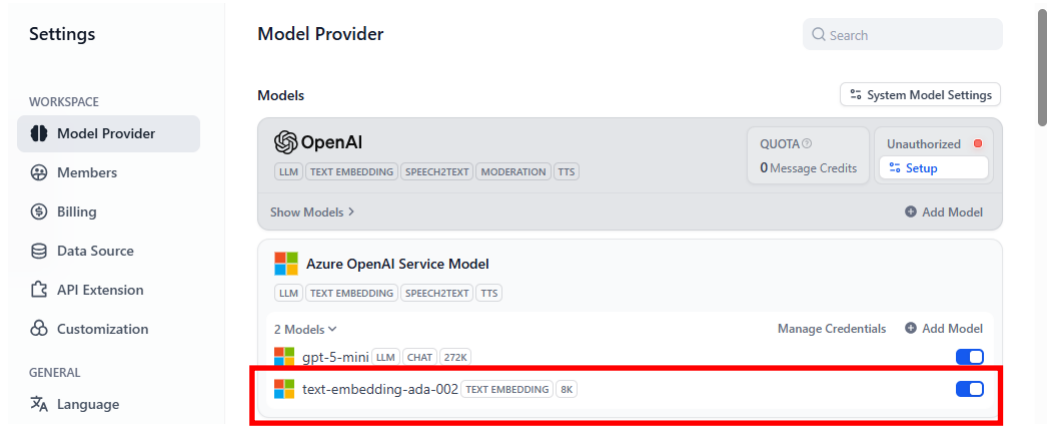
2.



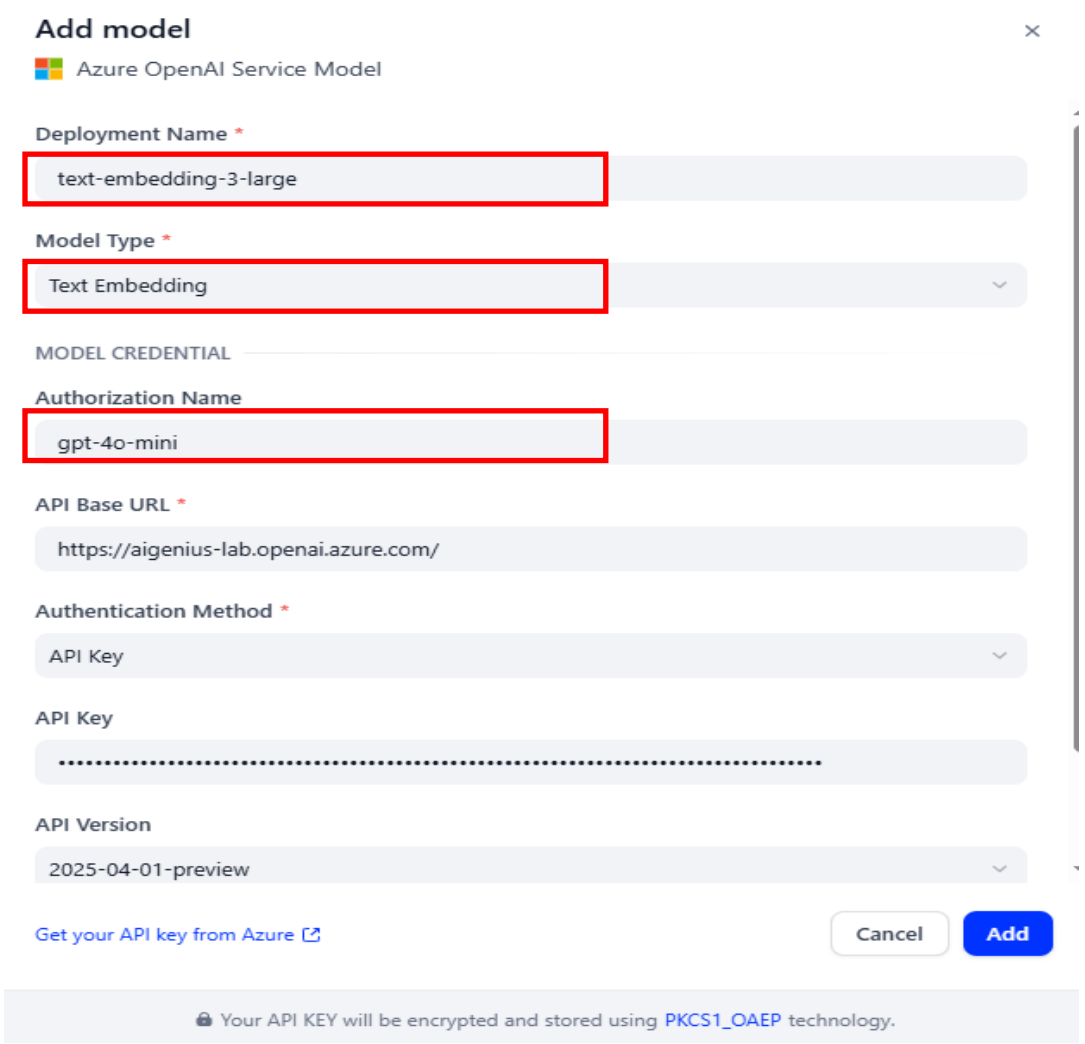
3.

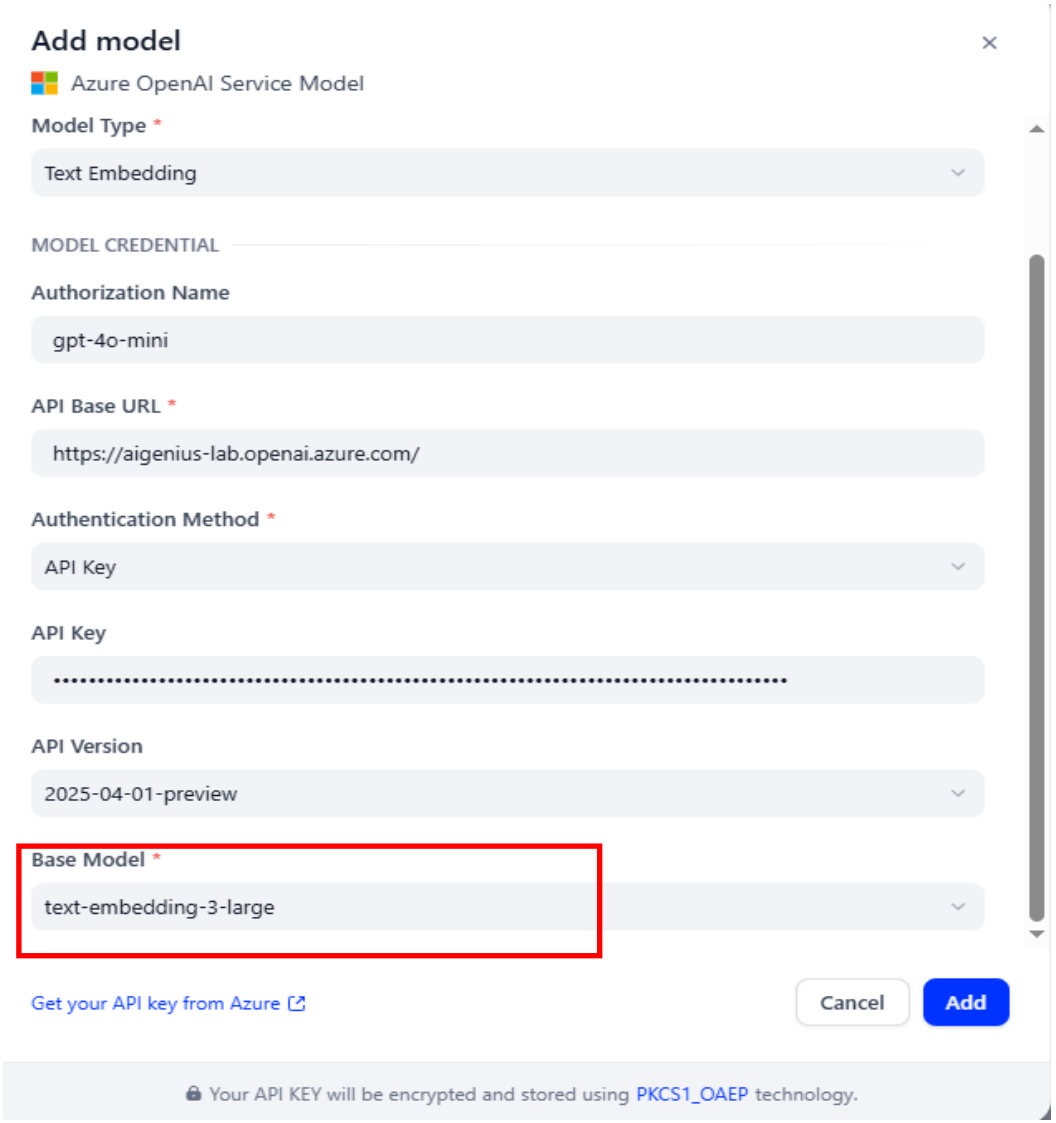


4.



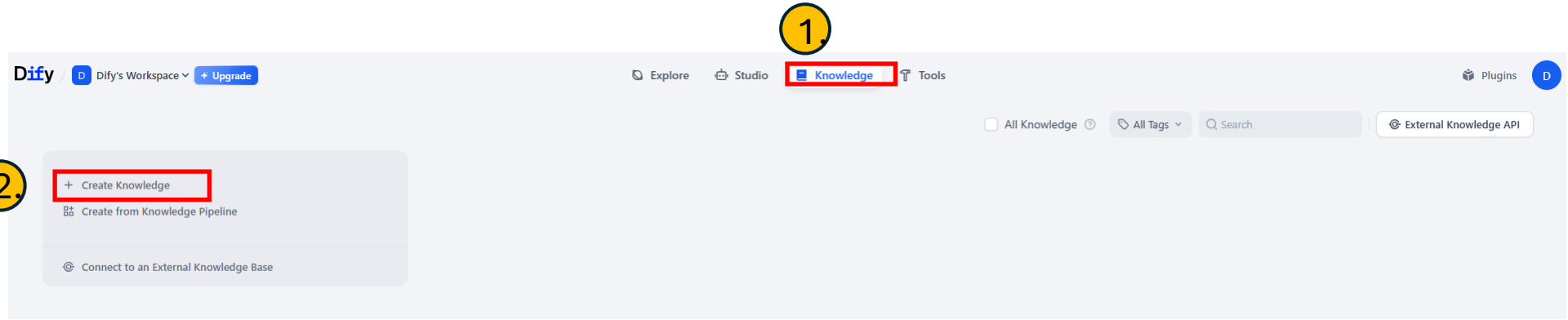
Embedding Model



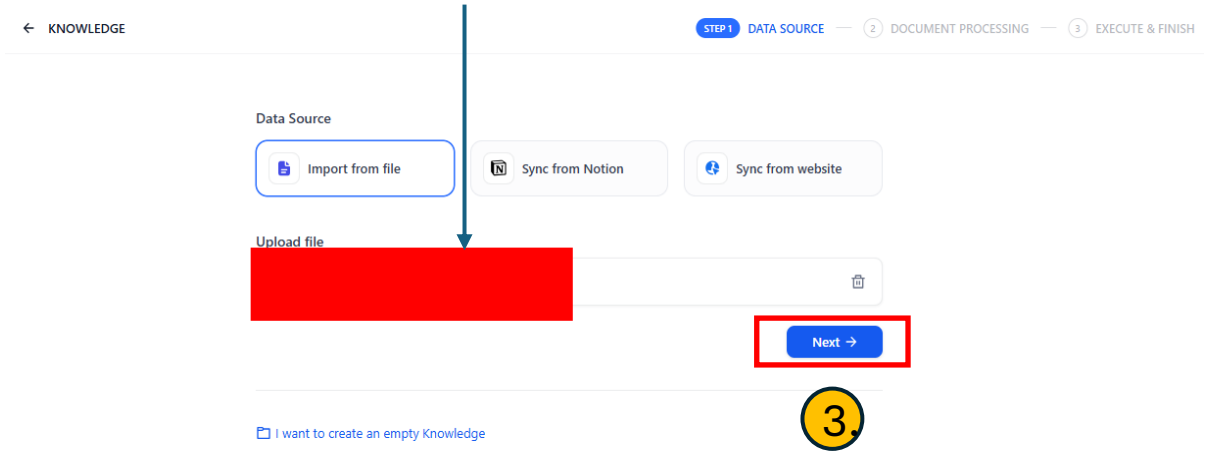


Create Knowledge Base

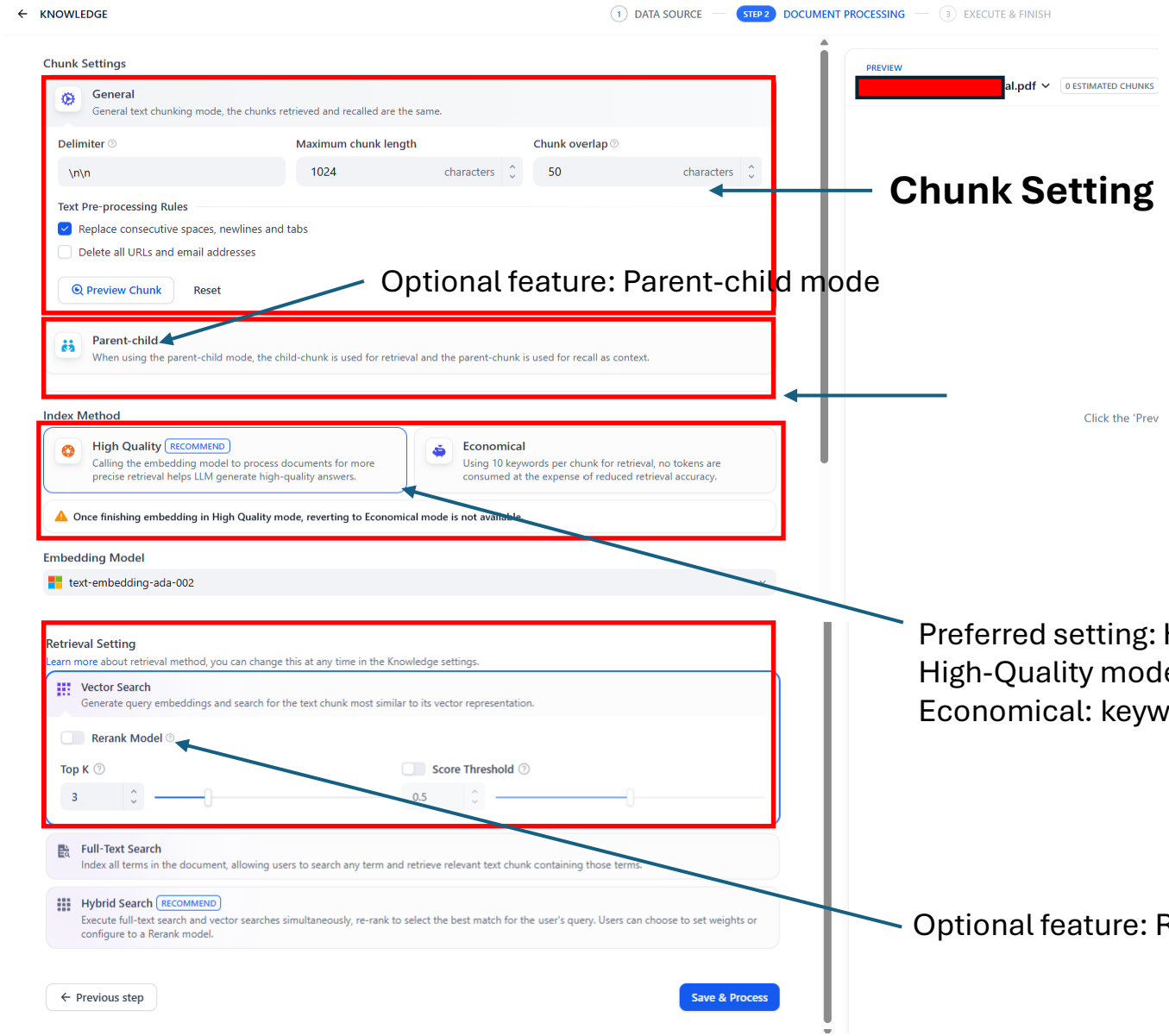
Create Knowledge Base-1/3



Select the document here



Create Knowledge Base-2/3



Chunk Setting

Optional feature: Parent-child mode

Preferred setting: High Quality
High-Quality mode: Vector Embedding mode
Economical: keyword indexing

Optional feature: Re-ranking model

There's no one-size-fits-all—test iteratively using the built-in Retrieval Testing tool

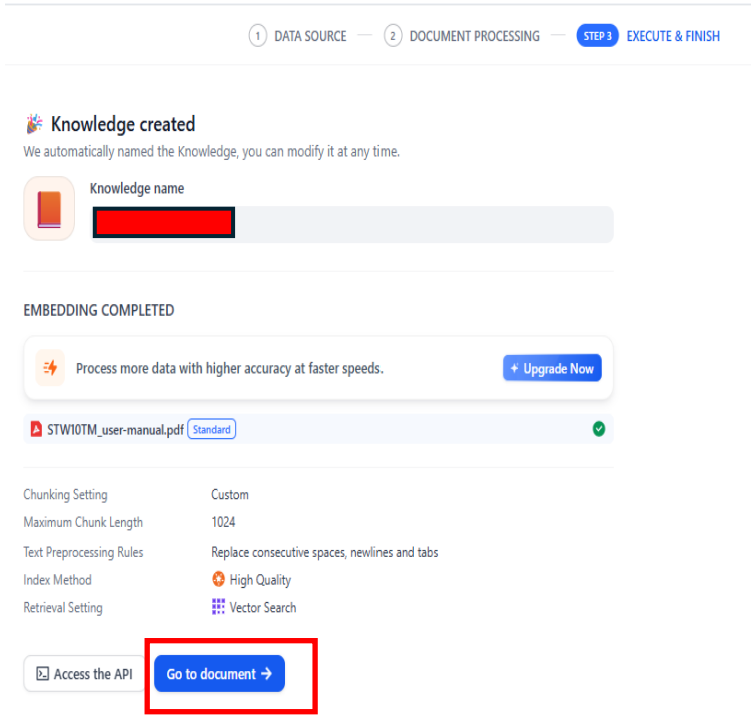
Chunk Size	Comment
Smaller (200-500 tokens)	Better for precise, granular retrieval (e.g., FAQs, code snippets, short answers). Reduces noise but risks losing broader context.
Medium (500-1000 tokens)	Good starting point for most text documents (articles, manuals). Balances precision and context.
Larger (1000-2000+ tokens)	For dense, narrative content needing more context (e.g., legal docs, long reports). But increases risk of irrelevant info (noise) and higher embedding costs.

Chunk Overlap

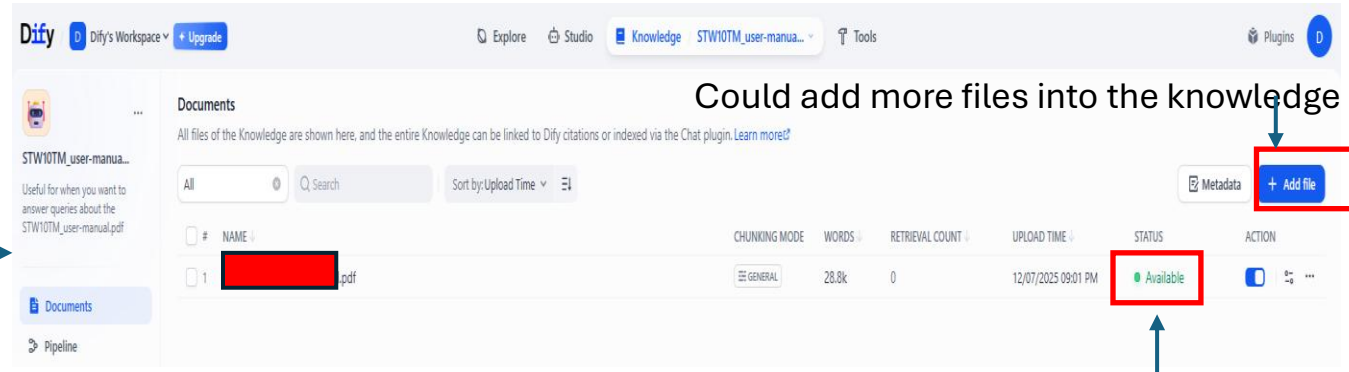
Purpose: Prevents information from being split awkwardly across chunk boundaries (e.g., a sentence cut in half loses meaning). Higher overlap improves context continuity but increases storage/processing costs slightly.

Create Knowledge Base-3/3

1.



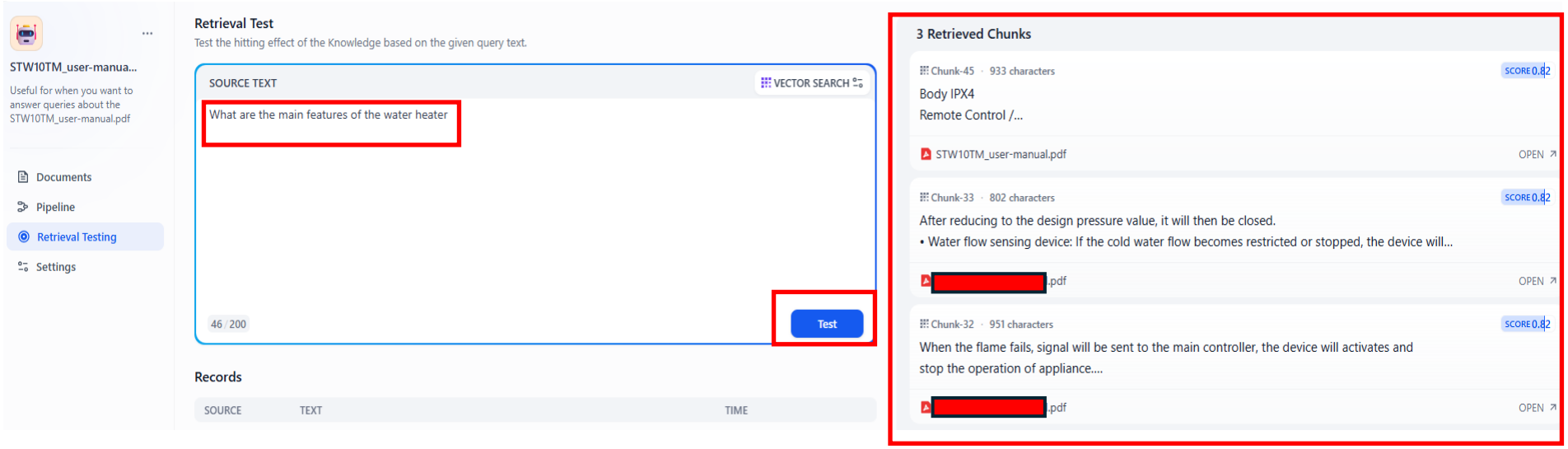
2.



Could add more files into the knowledge base

Status of the knowledge base

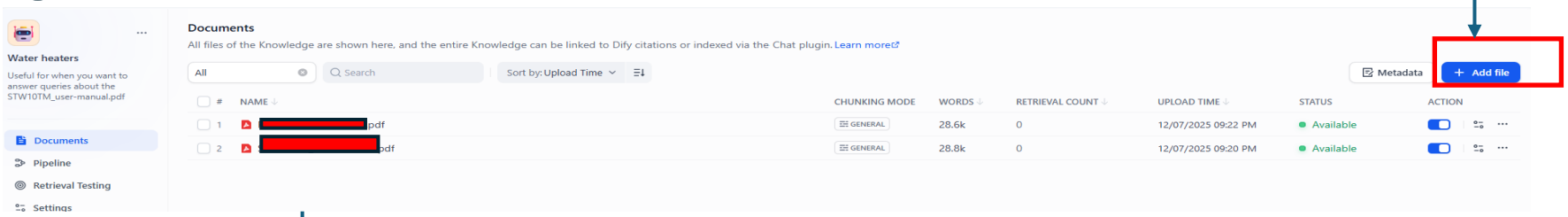
Retrieval Testing Tools



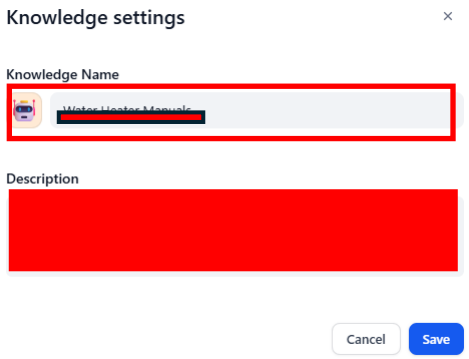
Finalize the setup of Knowledge Base

1. Adding more documents

Add more documents Here



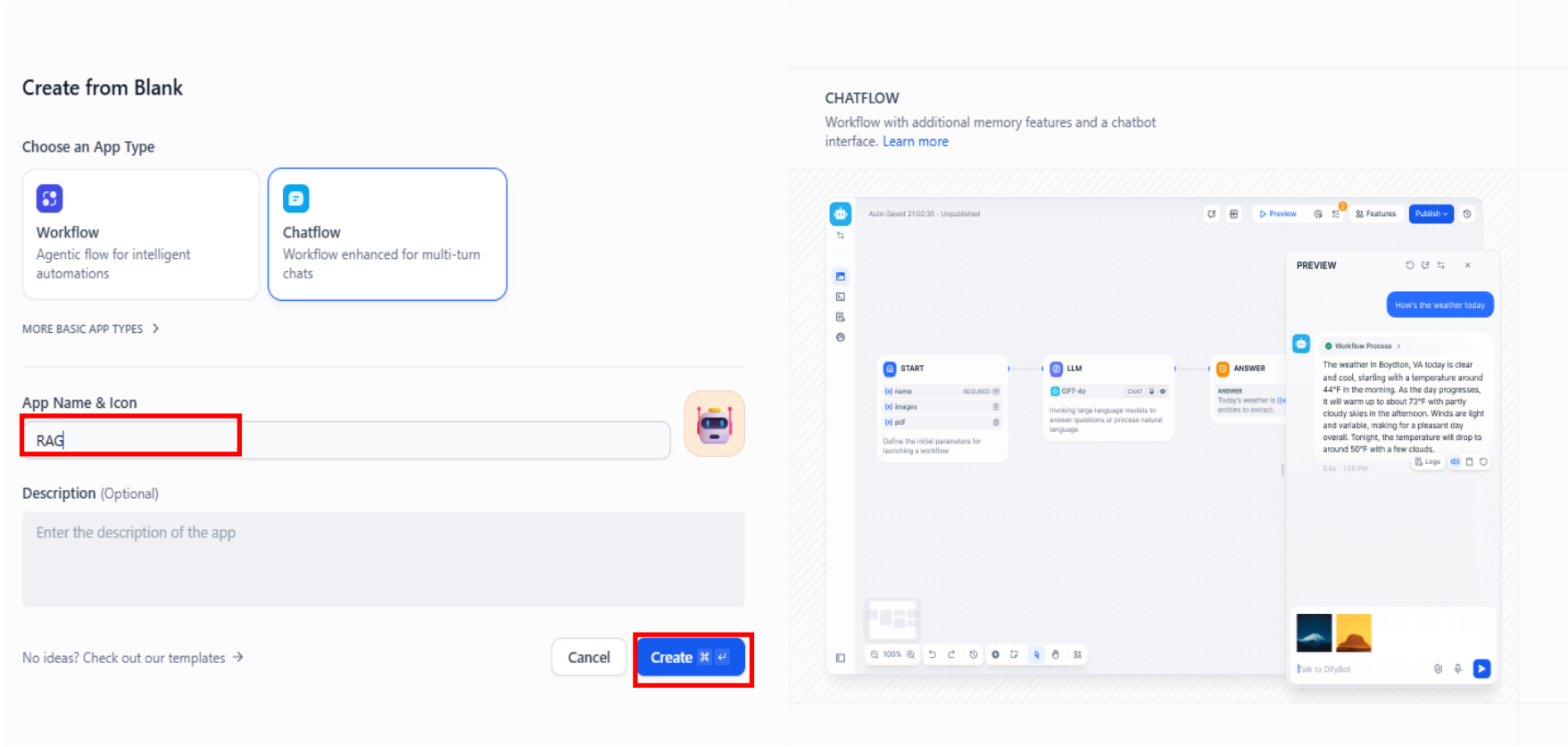
2.



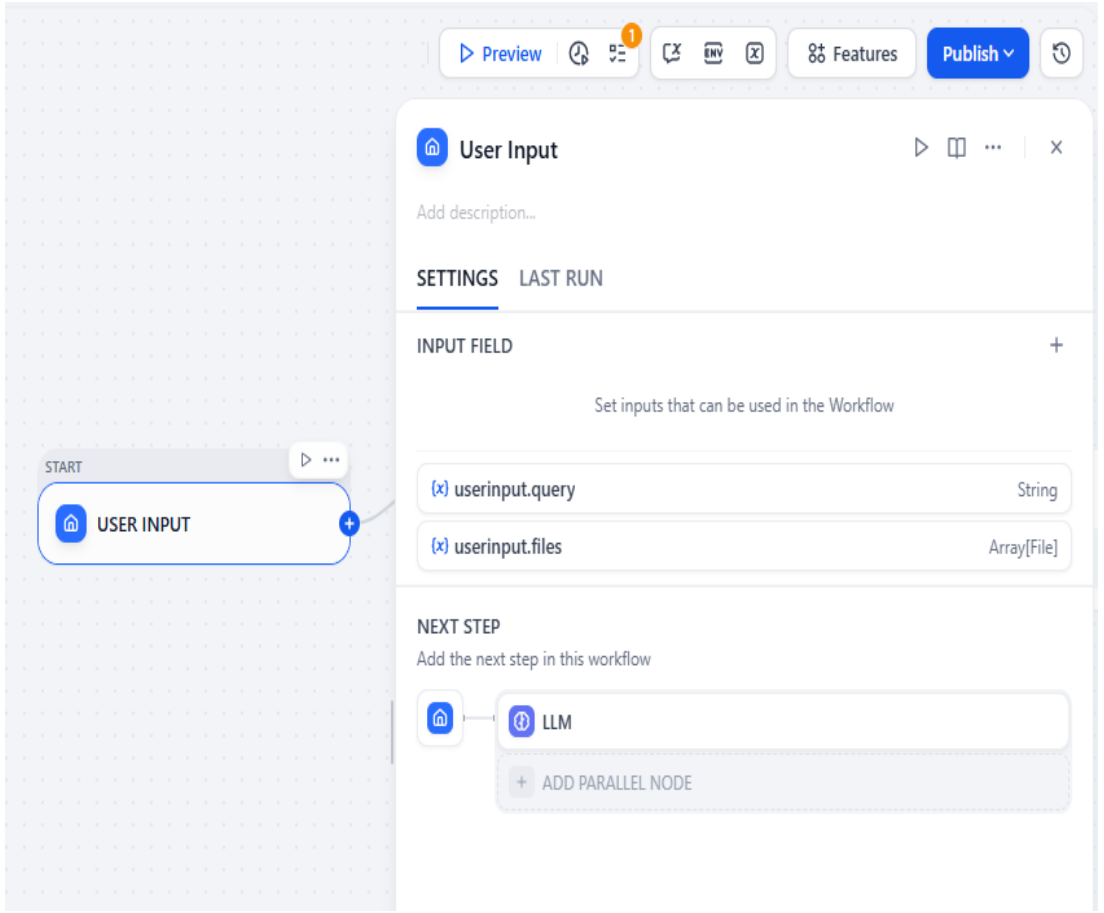
Revise the name and the Description of the knowledge base

Knowledge Retrieval

Create Chatflow

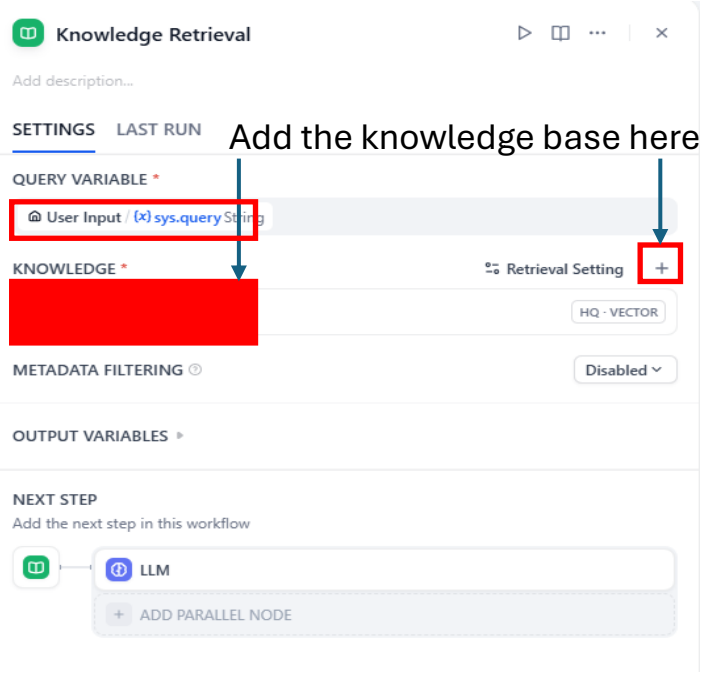


User Input


Preview
Features
Publish
User Input
Add description...
SETTINGS
LAST RUN
INPUT FIELD
Set inputs that can be used in the Workflow
START
USER INPUT
userinput.query
String
userinput.files
Array[File]
NEXT STEP
Add the next step in this workflow
LLM
ADD PARALLEL NODE

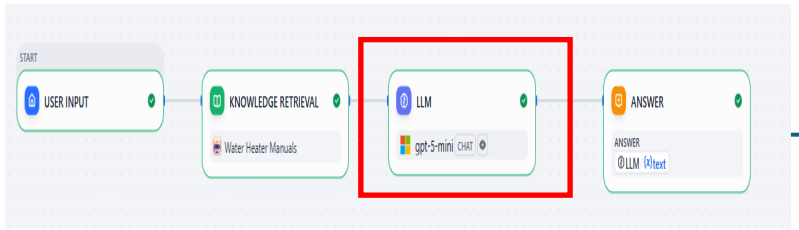
Knowledge Retrieval





LLM Setup

LLM Setup



Include the knowledge retrieval result here

LLM

Add description...

SETTINGS LAST RUN

MODEL *

gpt-4o-mini CHAT

CONTEXT

Knowledge Retrieval / (x) result Array[Object]

SYSTEM 494 Jinja (x)

Use the retrieved chunks to answer. Always check the "source" field in each chunk to know which product it refers to.

When comparing:
- Clearly state which product has which feature
- Use bullet points or tables
- If asked "which is better", give a balanced answer based only on the documents
- Always cite the source file name at the end of each claimIf the answer is not in the information below, say "I don't know".

Retrieved information:
Context

Question: User Input (x) query
Answer:

System prompt

System Prompt

SYSTEM 494 Jinja {x}

Use the retrieved chunks to answer. Always check the "source" field in each chunk to know which product it refers to.

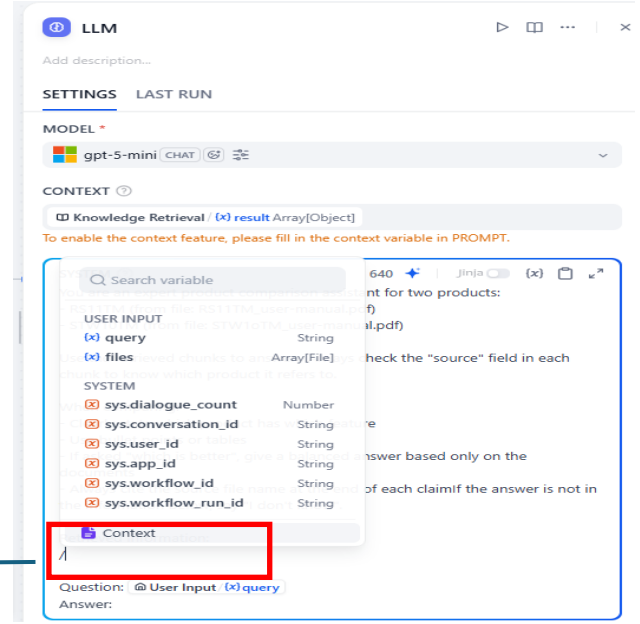
When comparing:
- Clearly state which product has which feature
- Use bullet points or tables
- If asked "which is better", give a balanced answer based only on the documents
- Always cite the source file name at the end of each claimIf the answer is not in the information below, say "I don't know".

Retrieved information:

Context

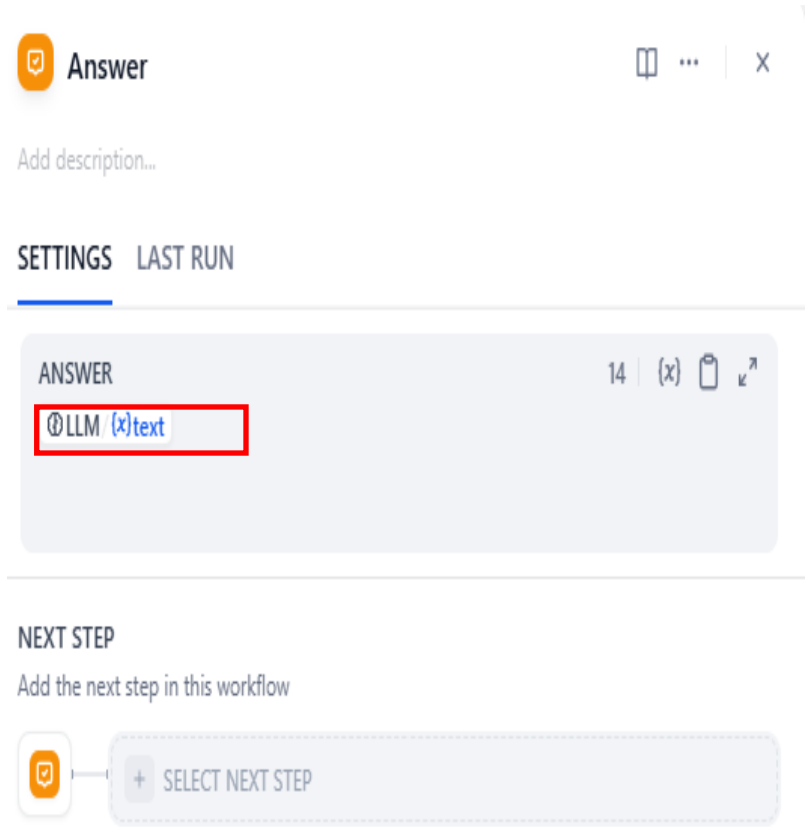
Question: User Input / (x) query

Answer:



Answer Node

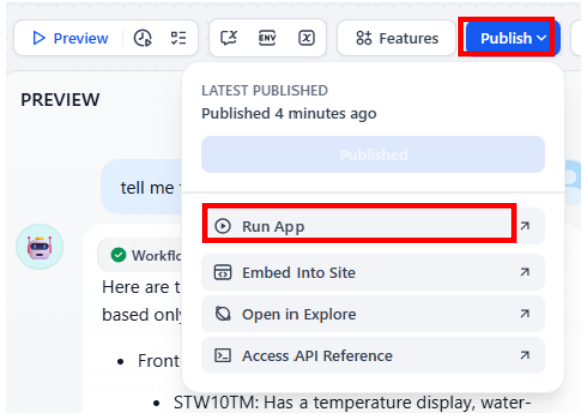
Answer Node

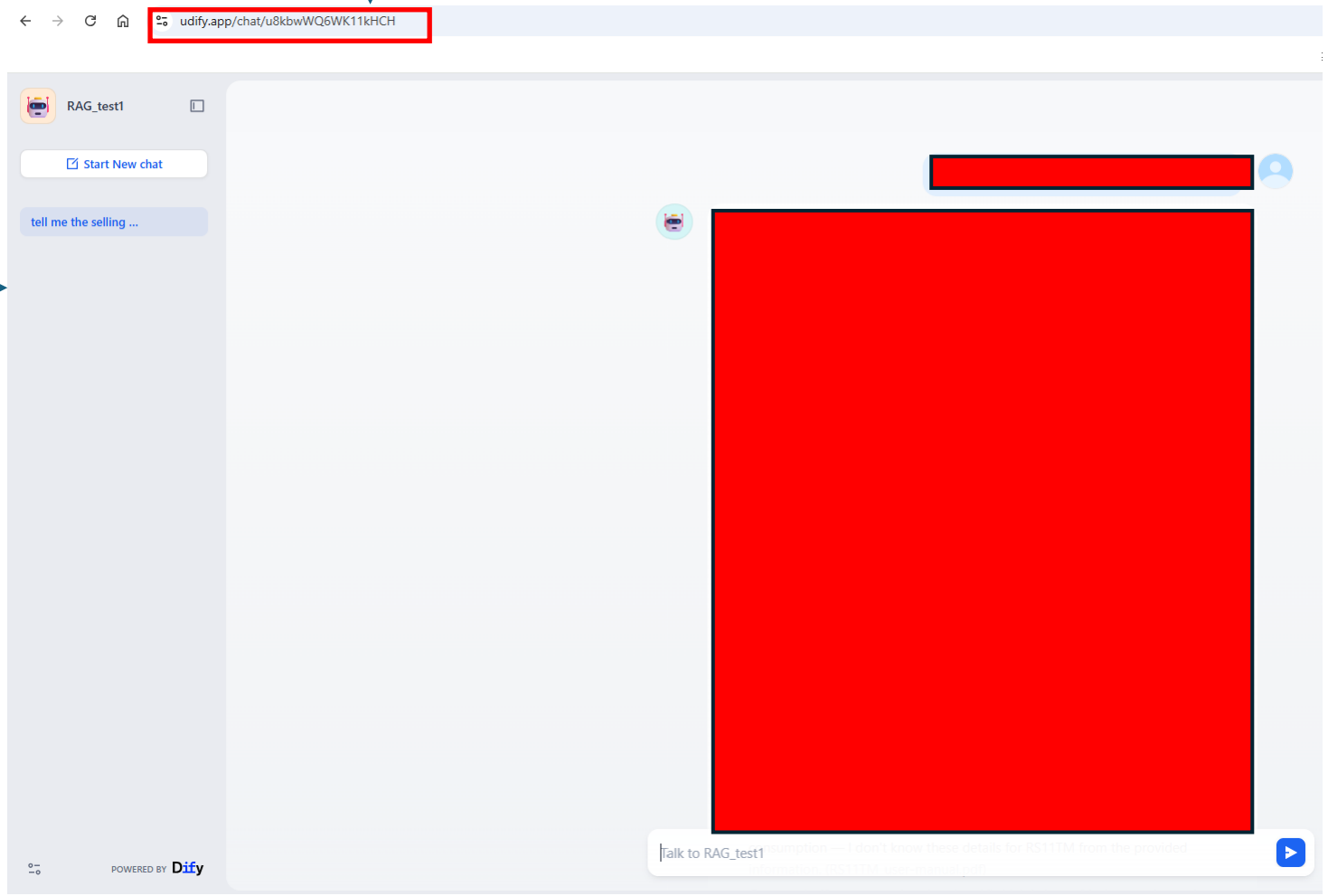

Answer
Add description...
SETTINGS
LAST RUN
ANSWER
14
{x}
LLM
{x}text
NEXT STEP
Add the next step in this workflow
SELECT NEXT STEP

Publish the Chatbot

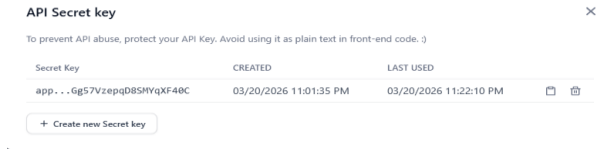
Publish the Chatbot

URL of the Chatbot

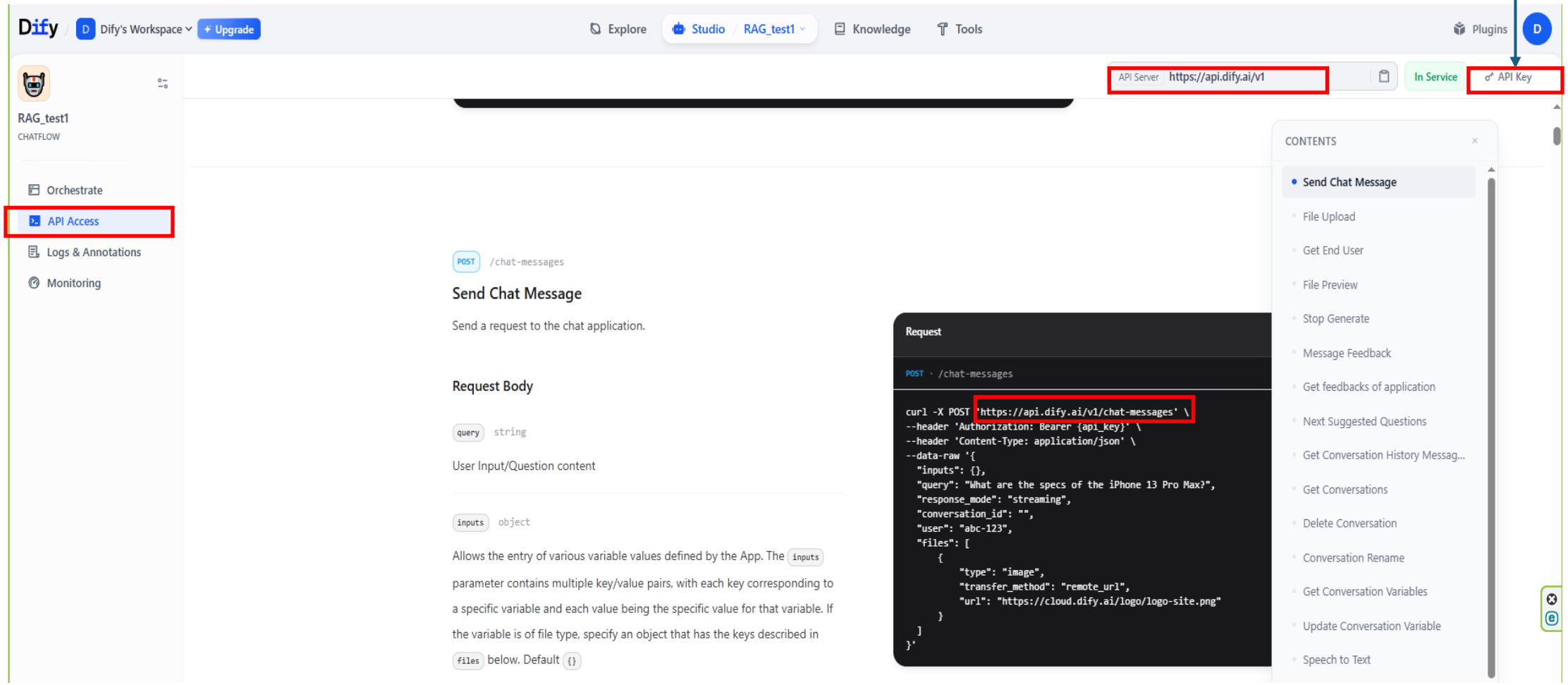




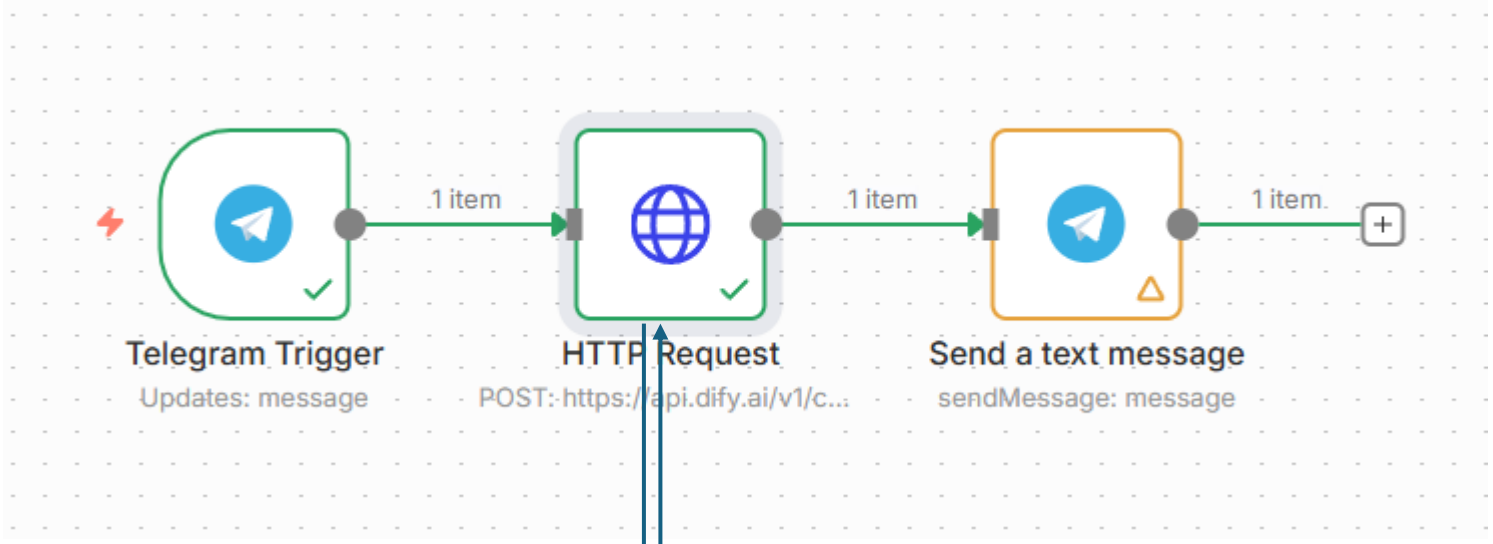
API Key



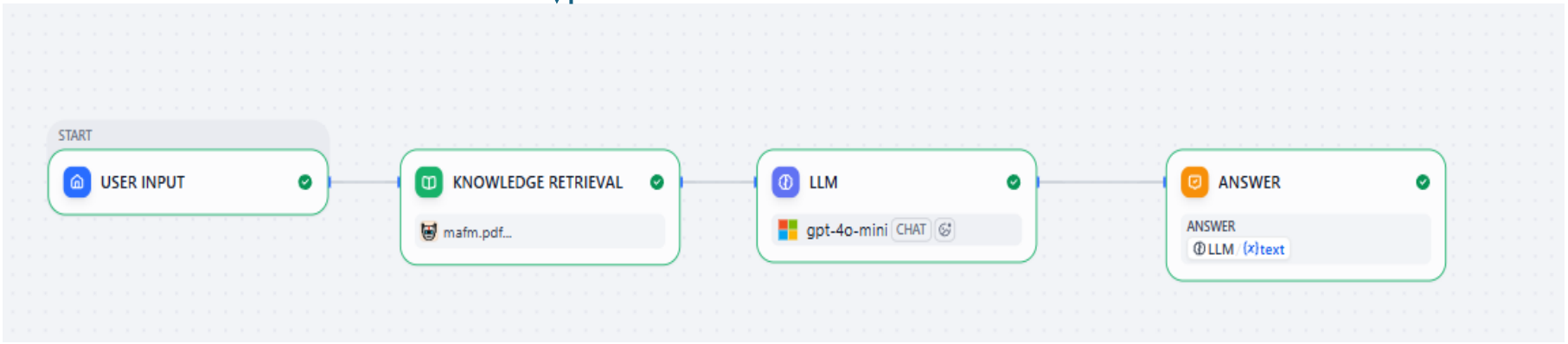
Create the API key here



Deploy RAG Chatbot with Telegram

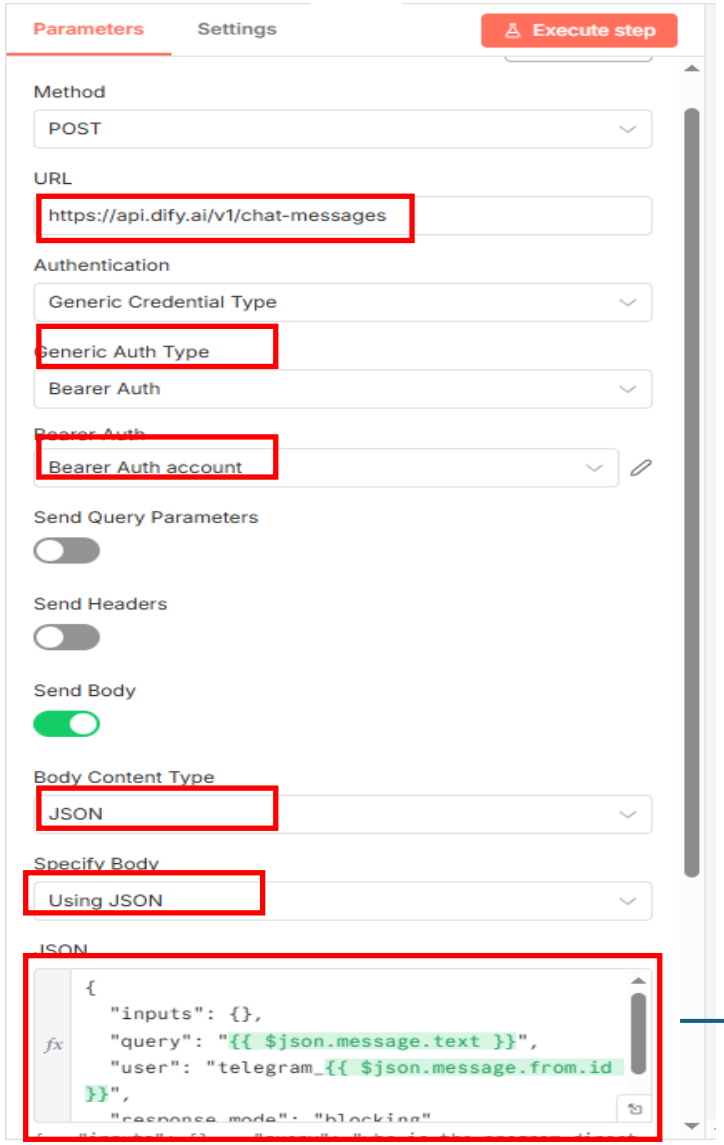


N8n workflow



Dify workflow

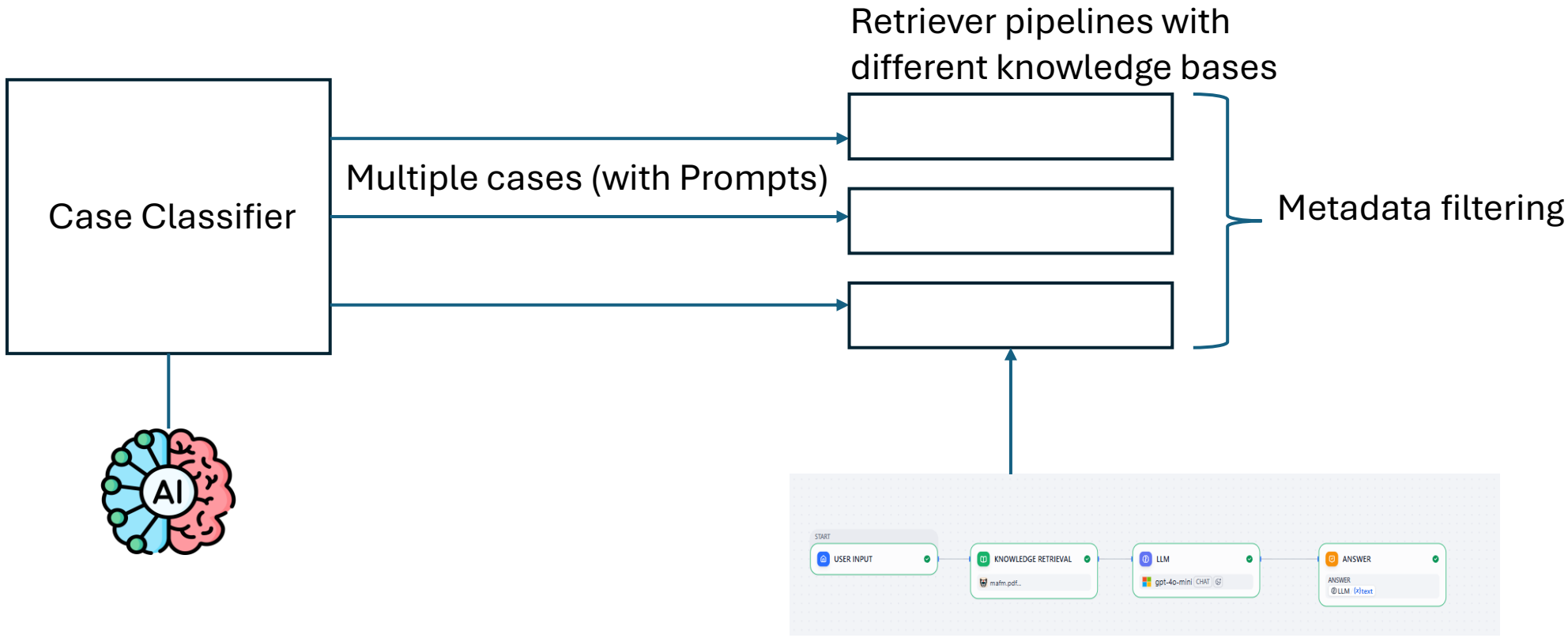
HTTP Request Node



JSON Content

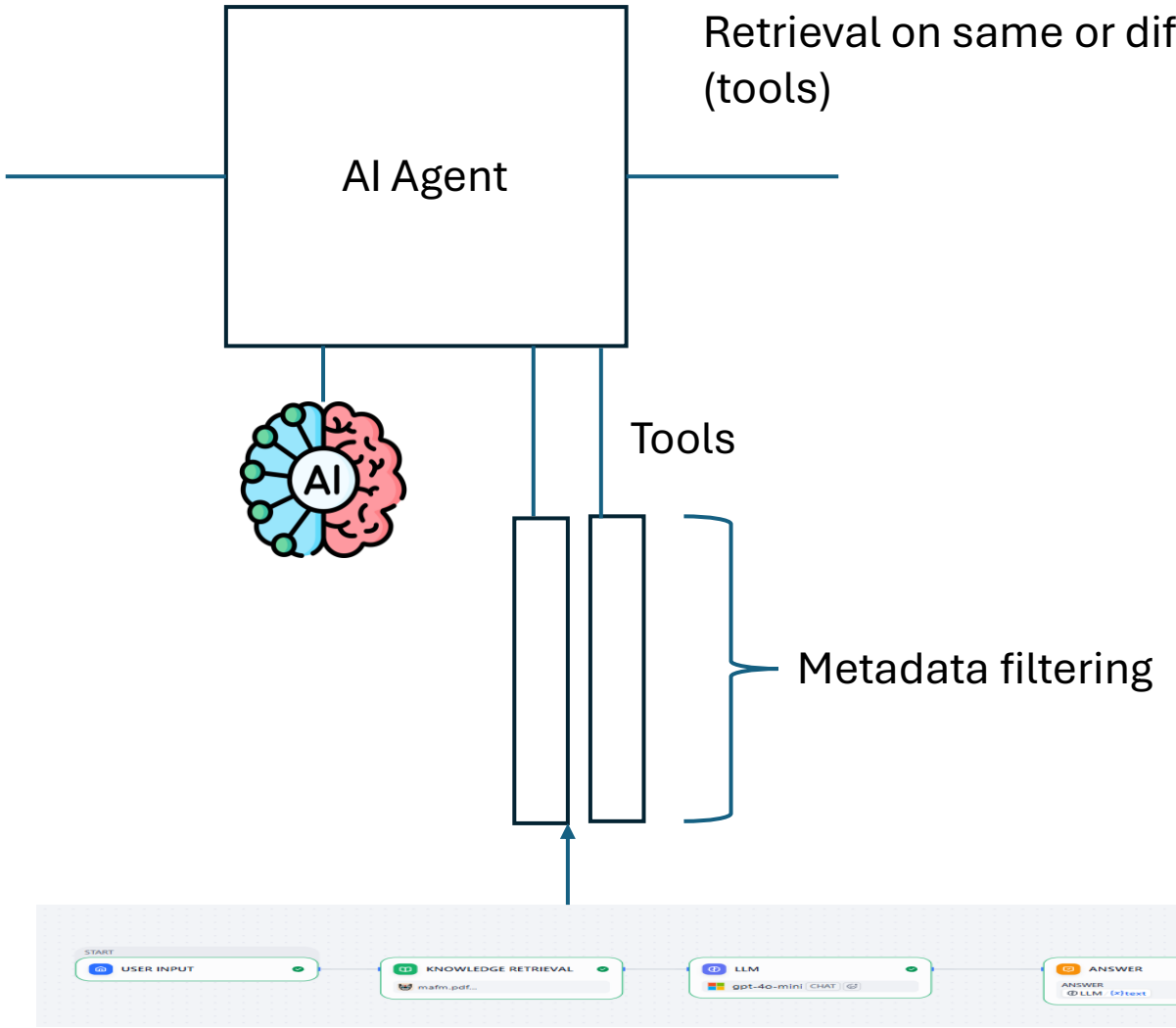
```
{
  "inputs": {},
  "query": "{{ $json.message.text }}",
  "user": "telegram_{{ $json.message.from.id }}",
  "response_mode": "blocking"
}
```

Sample Workflow Design for RAG



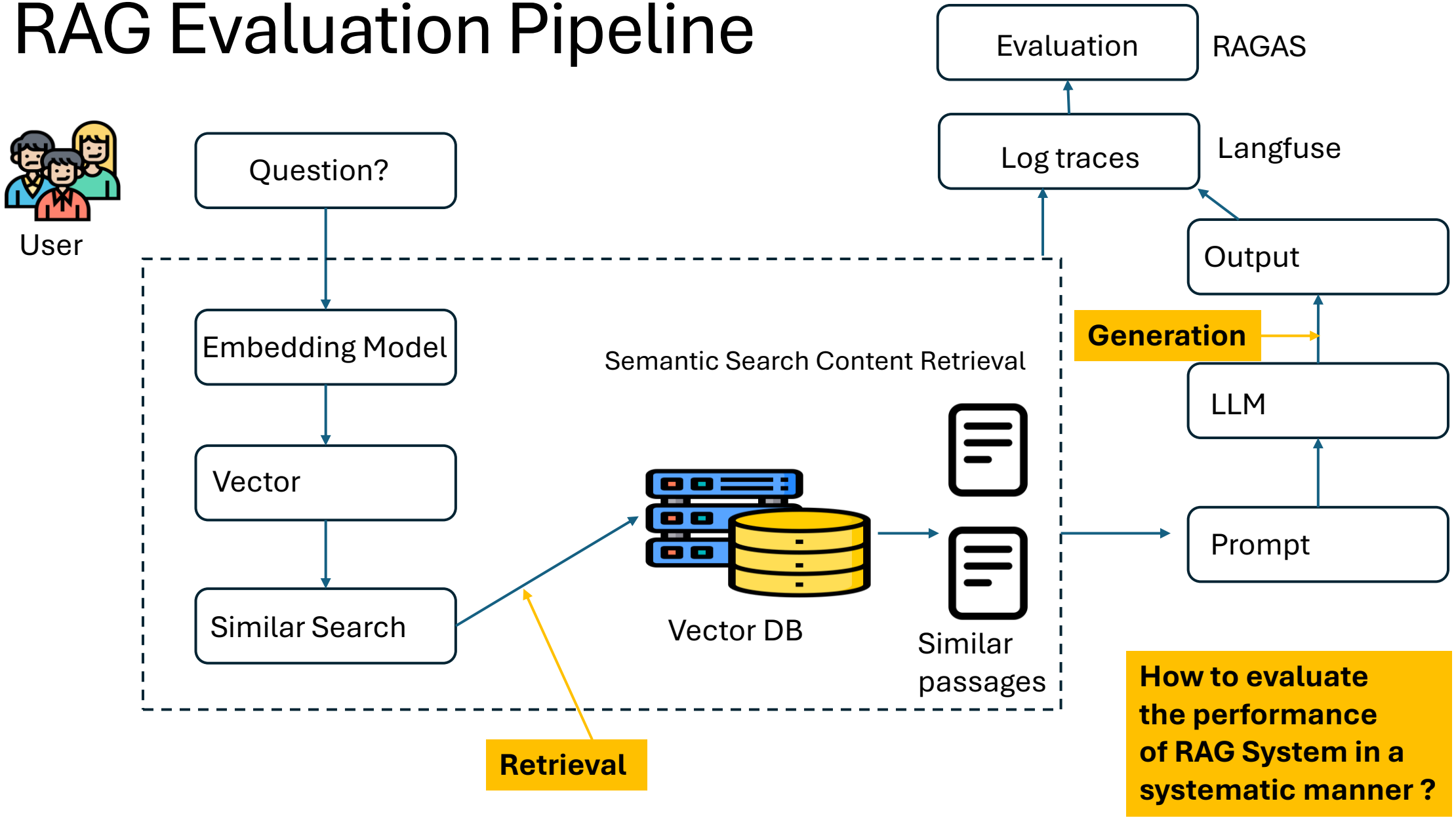
Sample Workflow Design for RAG

Could support more complicated queries
e.g. Queries which need multiple recursive Retrieval on same or different retrieval pipelines (tools)

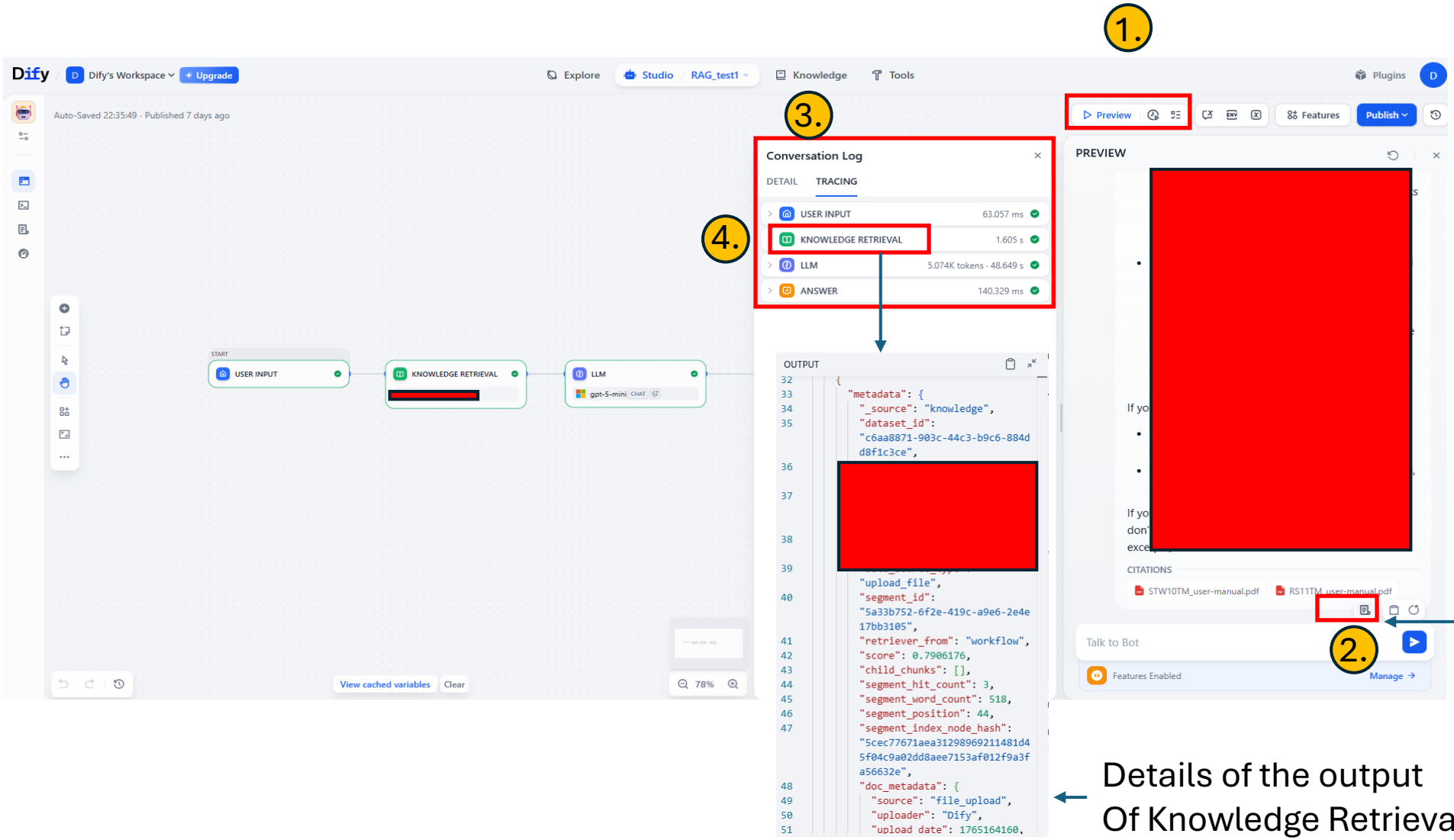


RAG Evaluation Pipeline

RAG Evaluation Pipeline



Logs and Annotations



Click here to show the log

Details of the output
Of Knowledge Retrieval

Evaluation Metrics For RAG Systems-1/2

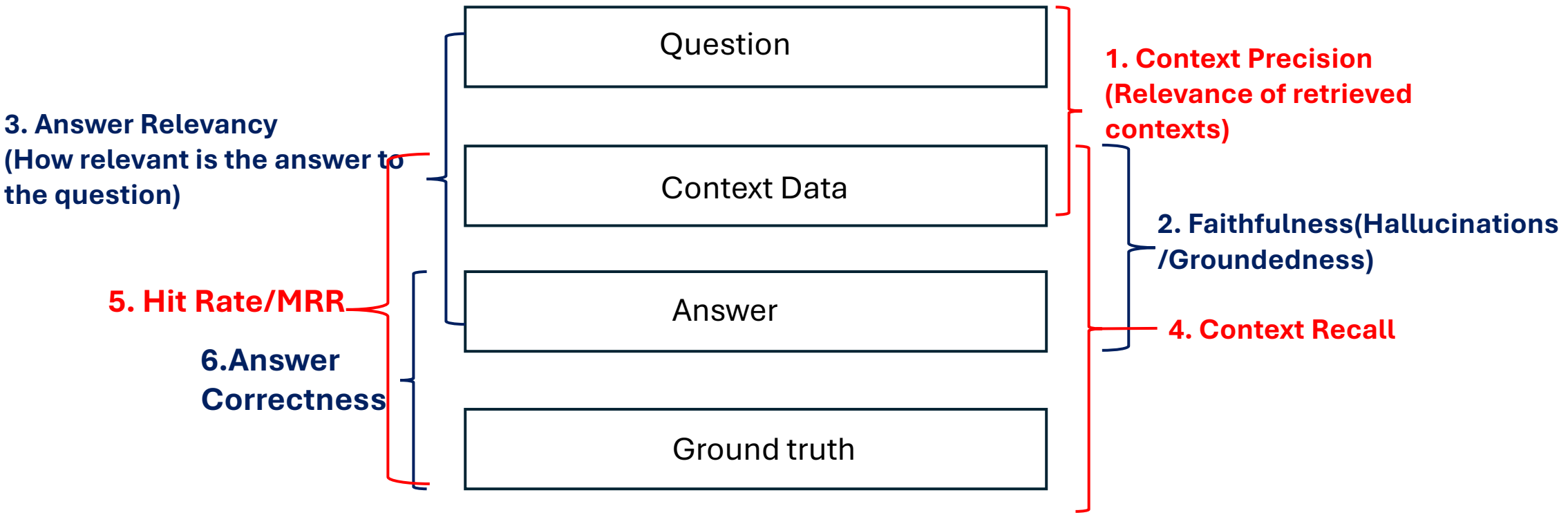
Retrieval Metrics

- Precision
- Recall
- Hit Rate
- Mean Reciprocal Rank (MRR)

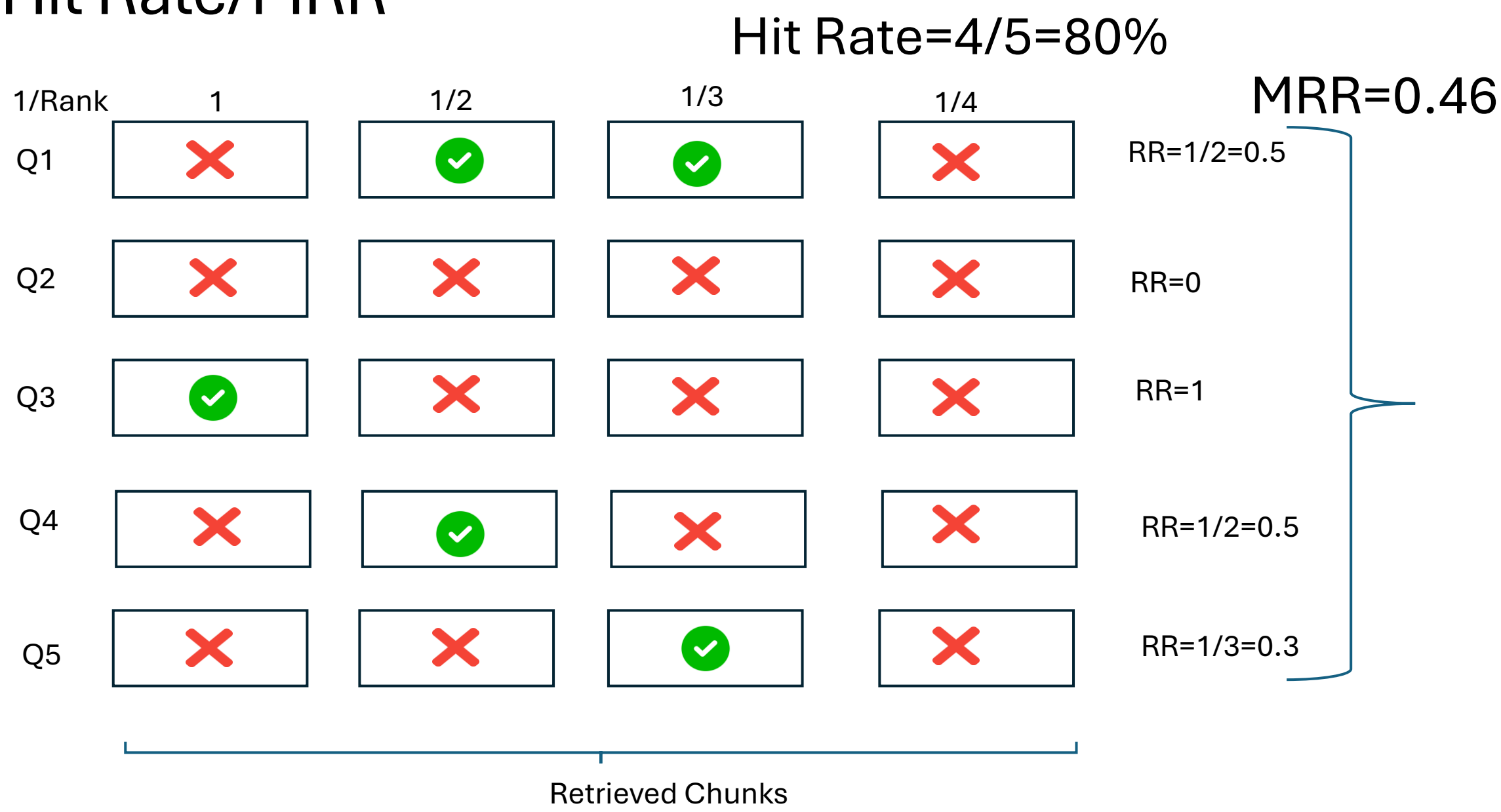
Generation Metrics

- Faithfulness
- Answer Relevancy
- Answer Correctness

Evaluation Metrics For RAG Systems-2/2



Hit Rate/MRR


Hit Rate=4/5=80%
MRR=0.46
1/Rank
1
1/2
1/3
1/4
Q1
Q2
Q3
Q4
Q5
RR=1/2=0.5
RR=0
RR=1
RR=1/2=0.5
RR=1/3=0.3
Retrieved Chunks

Query with Relational Database

Data Retrieval with Relational DB and LLM



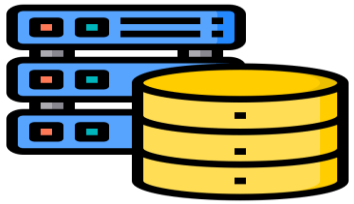
Question?

Prompt

LLM

Generated SQL

Response



Relational DB

product_price1

Name text	Price int8
ALPHA-7S	3200
APEX-PREDATOR	35900
AURORA-LITE	6100
BLADE-RUNNER	19900
COSMICRAY	5500
CRYSTALVIEW	8900
CYBERKNIGHT	19200
DRAGONFIRE	29800
ECLIPSE-X	15400

Table used in the Class exercise

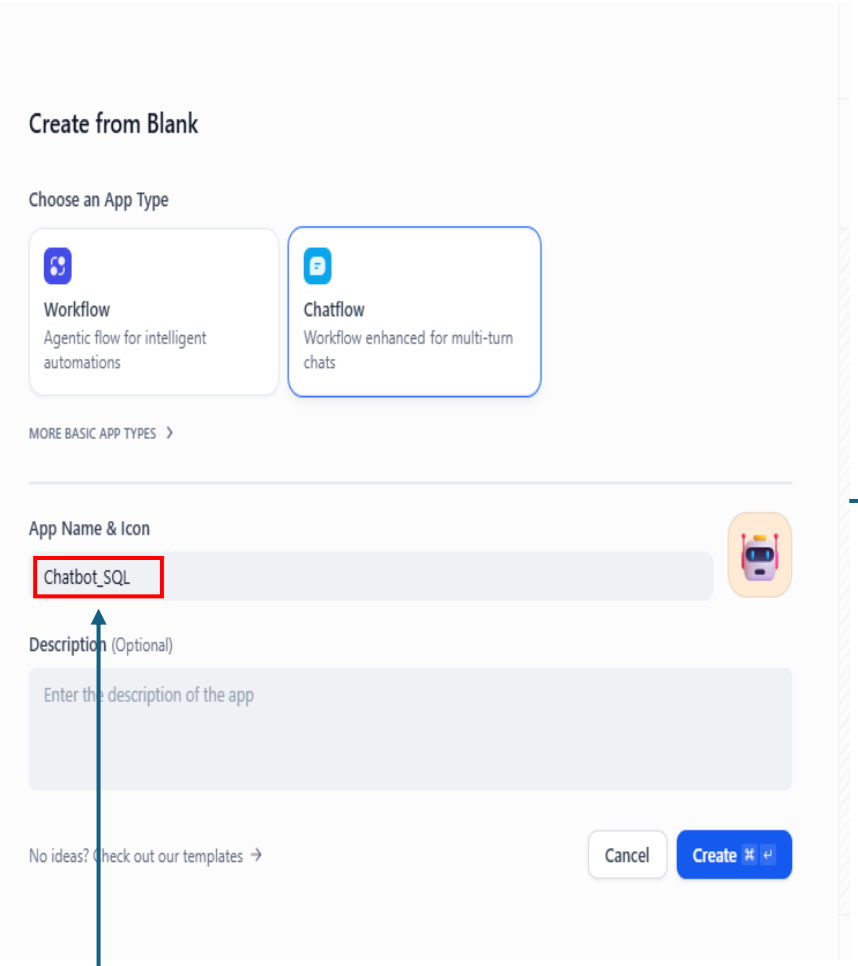
Class Exercise Two

RAG with Relational DB

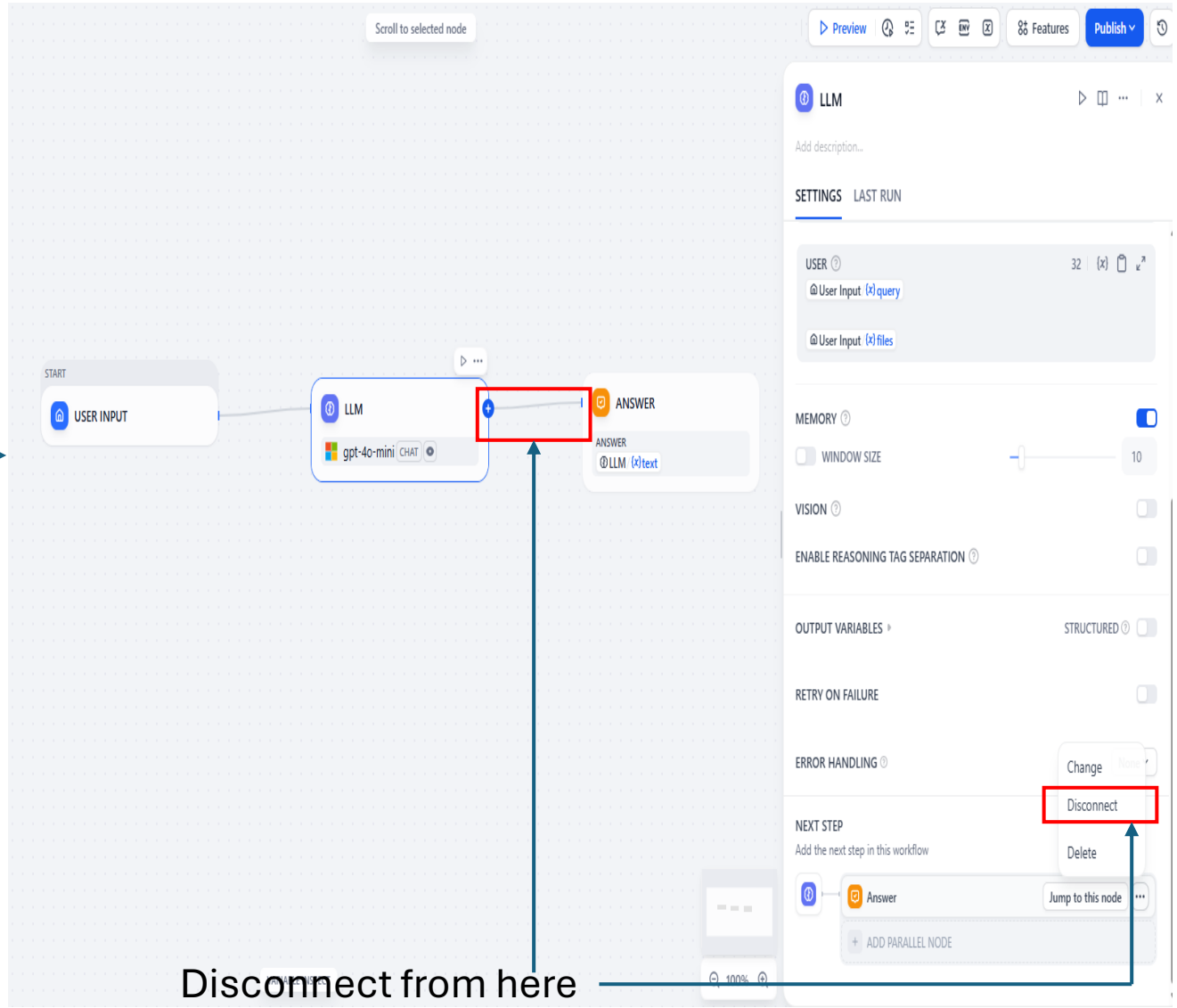
Overview


Input the user query here
Generate the SQL query based on the user input.
To fetch the data from the SQL DB based on the SQL query
Convert the result as text file (which will be used for generate the final response with LLM)
Generate the final response from LLM
To response to the user
START
USER INPUT
SQL_GENERATION
gpt-5-mini
CHAT
SQL DATABASE ACCESS
DATA PROCESSING
RESPONSE_GENERATE
gpt-5-mini
CHAT
ANSWER
ANSWER
LLM 2
{x} text

Create the Chatflow

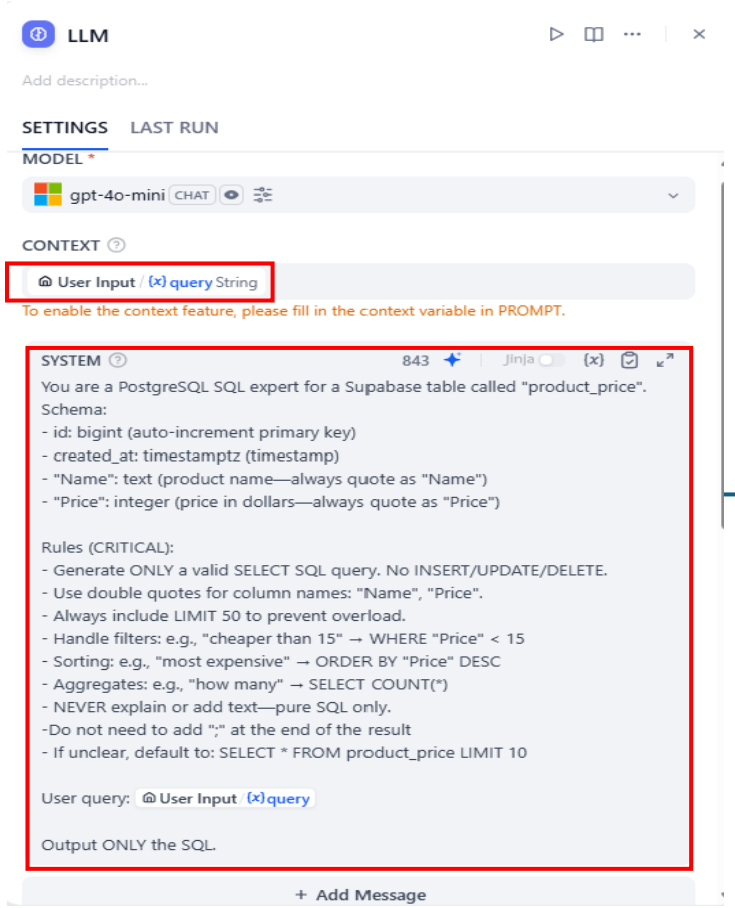


Define the name of the chatflow



Disconnect from here

LLM Setup



System Prompt

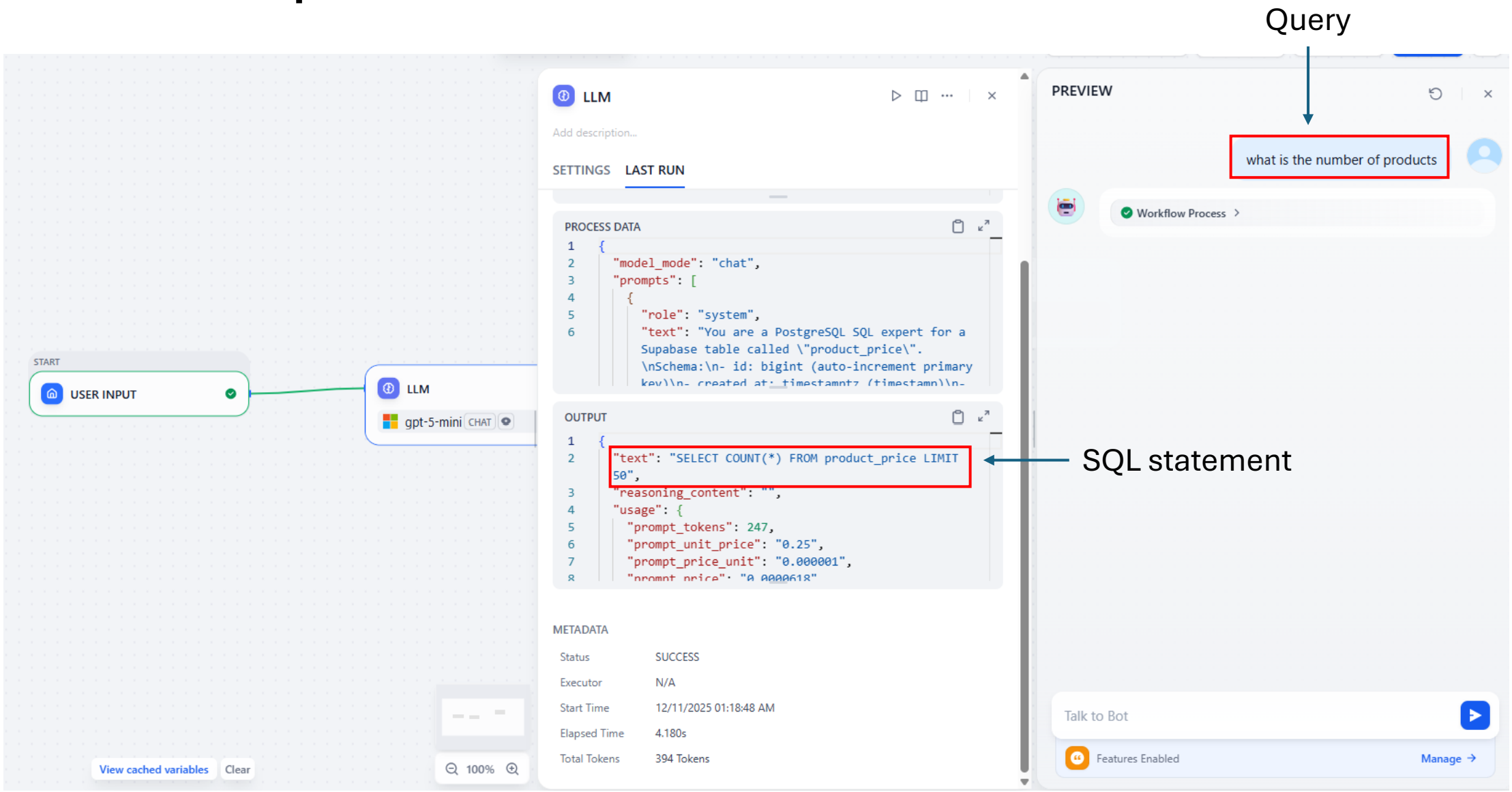
You are a PostgreSQL SQL expert for a Supabase table called "product_price".
Schema:
- id: bigint (auto-increment primary key)
- created_at: timestamptz (timestamp)
- "Name": text (product name—always quote as "Name")
- "Price": integer (price in dollars—always quote as "Price")

Rules (CRITICAL):
- Generate ONLY a valid SELECT SQL query. No INSERT/UPDATE/DELETE.
- Use double quotes for column names: "Name", "Price".
- Always include LIMIT 50 to prevent overload.
- Handle filters: e.g., "cheaper than 15" → WHERE "Price" < 15
- Sorting: e.g., "most expensive" → ORDER BY "Price" DESC
- Aggregates: e.g., "how many" → SELECT COUNT(*)
- NEVER explain or add text—pure SQL only.
-Do not need to add ";" at the end of the result
- If unclear, default to: SELECT * FROM product_price LIMIT 10

User query: {{#sys.query#}}

Output ONLY the SQL.

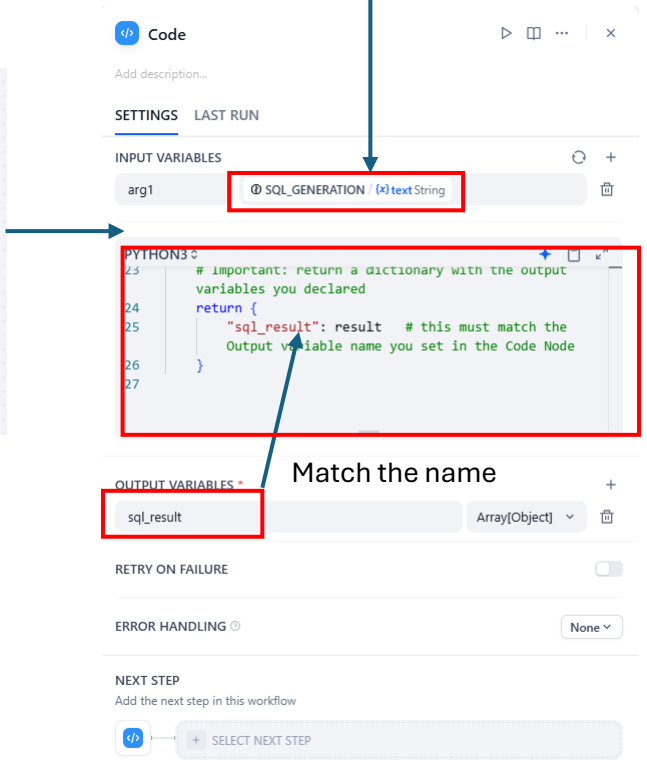
LLM Setup


Query
START
USER INPUT
LLM
gpt-5-mini CHAT
LLM
Add description...
SETTINGS
LAST RUN
PROCESS DATA
{
"model_mode": "chat",
"prompts": [
{
"role": "system",
"text": "You are a PostgreSQL SQL expert for a Supabase table called \"product_price\". \nSchema:\n- id: bigint (auto-increment primary
OUTPUT
{
"text": "SELECT COUNT(*) FROM product_price LIMIT 50",
"reasoning_content": "",
"usage": {
"prompt_tokens": 247,
"prompt_unit_price": "0.25",
"prompt_price_unit": "0.000001",
SQL statement
METADATA
Status SUCCESS
Executor N/A
Start Time 12/11/2025 01:18:48 AM
Elapsed Time 4.180s
Total Tokens 394 Tokens
View cached variables
Clear
100%
PREVIEW
what is the number of products
Workflow Process
Talk to Bot
Features Enabled
Manage

Code Node

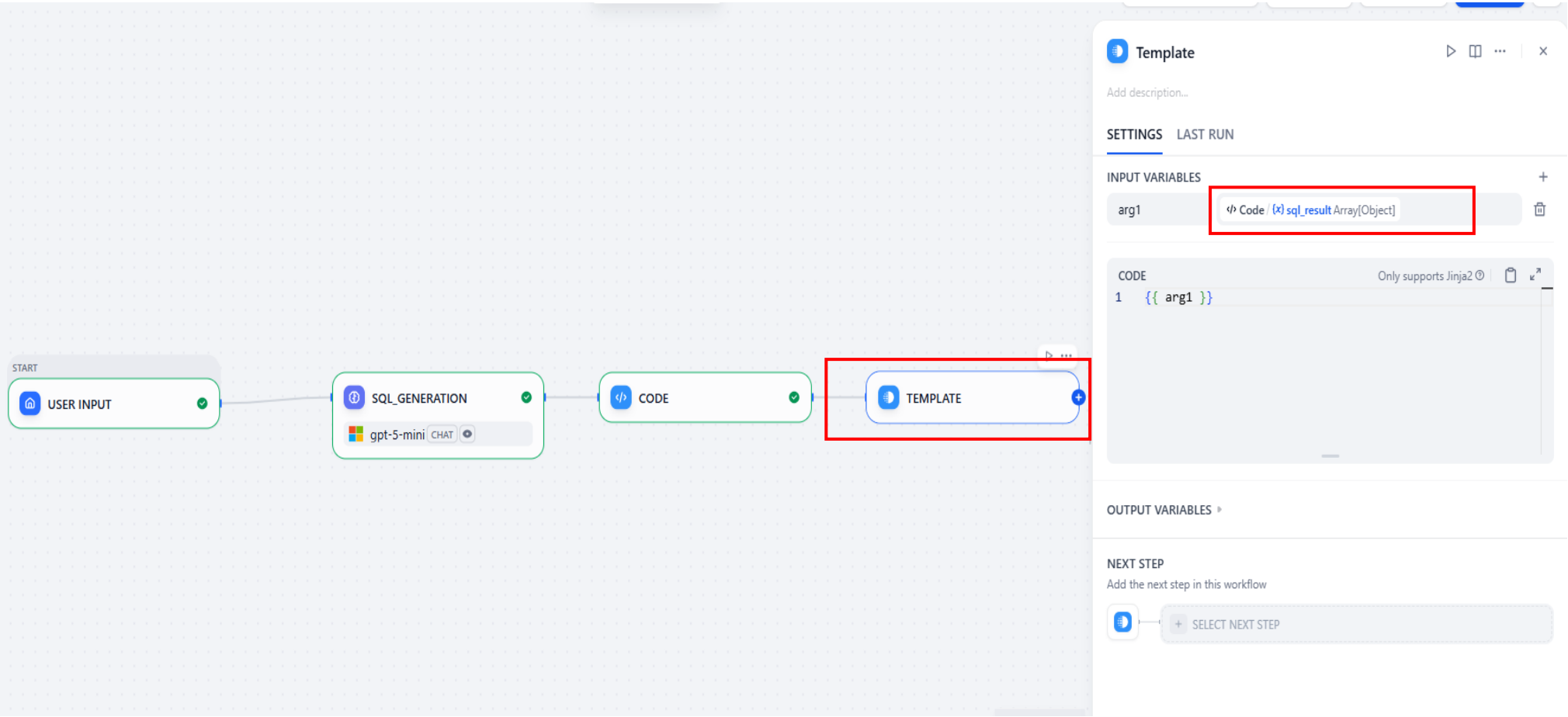


Result from previous LLM node

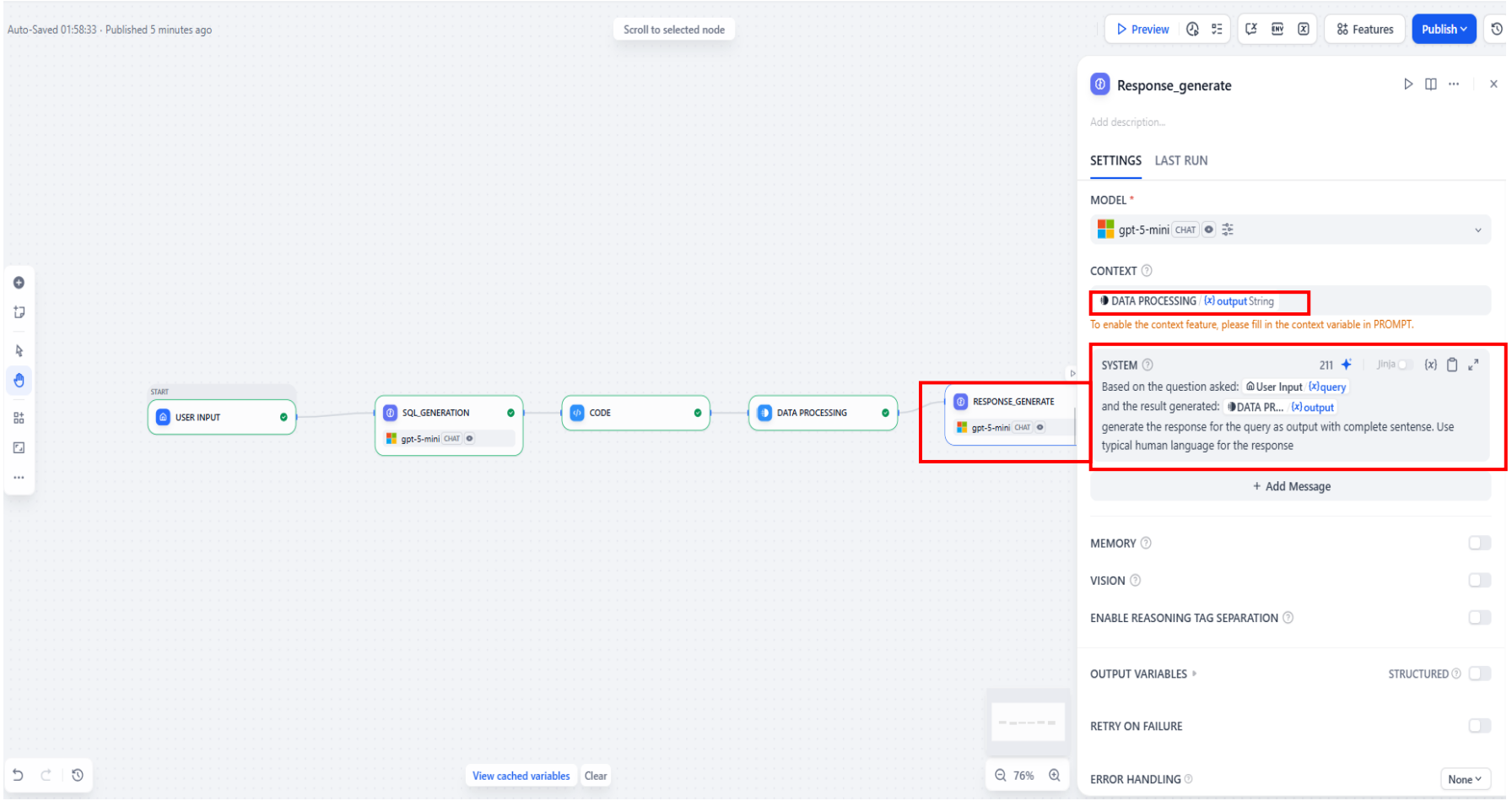


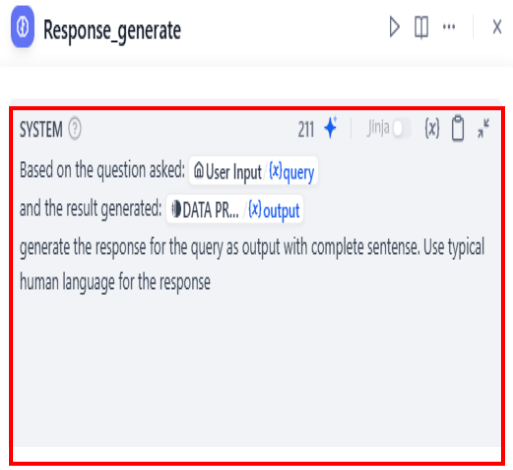


Template Node



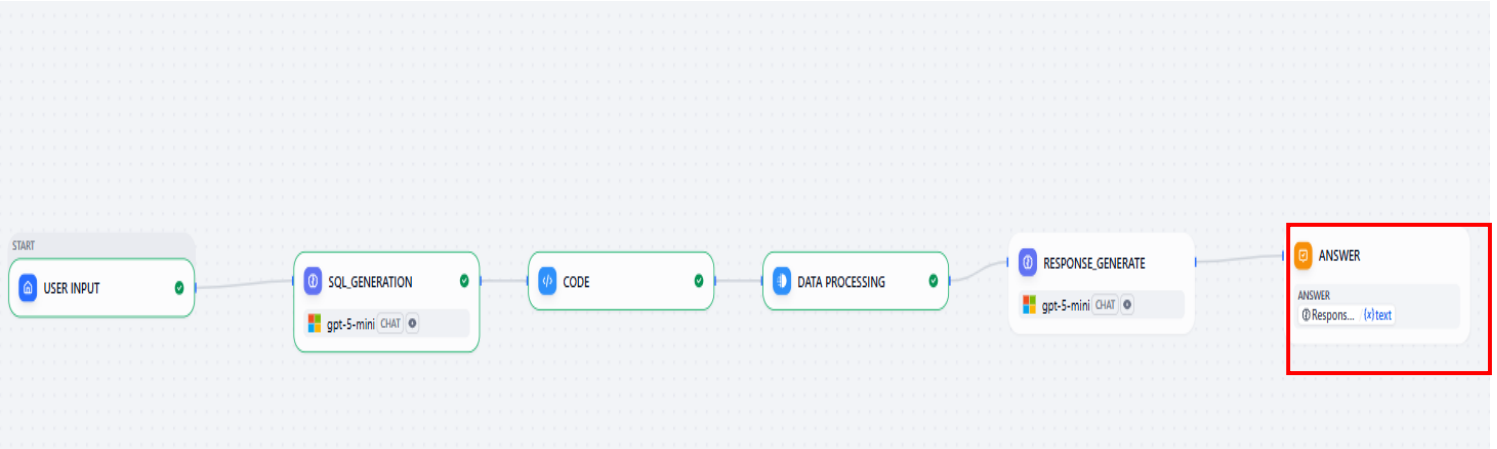
LLM Node (Generate Response)

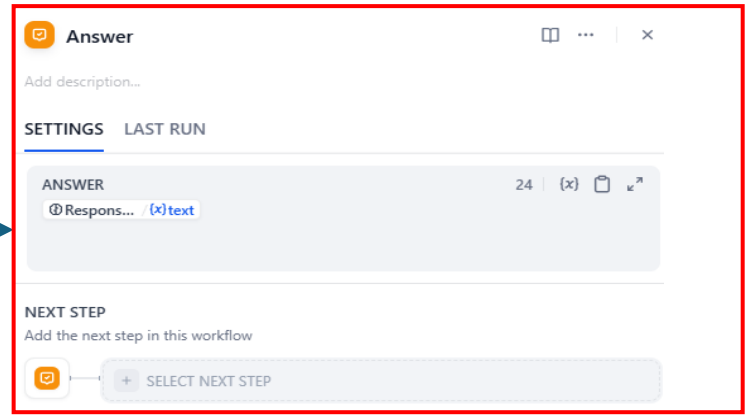




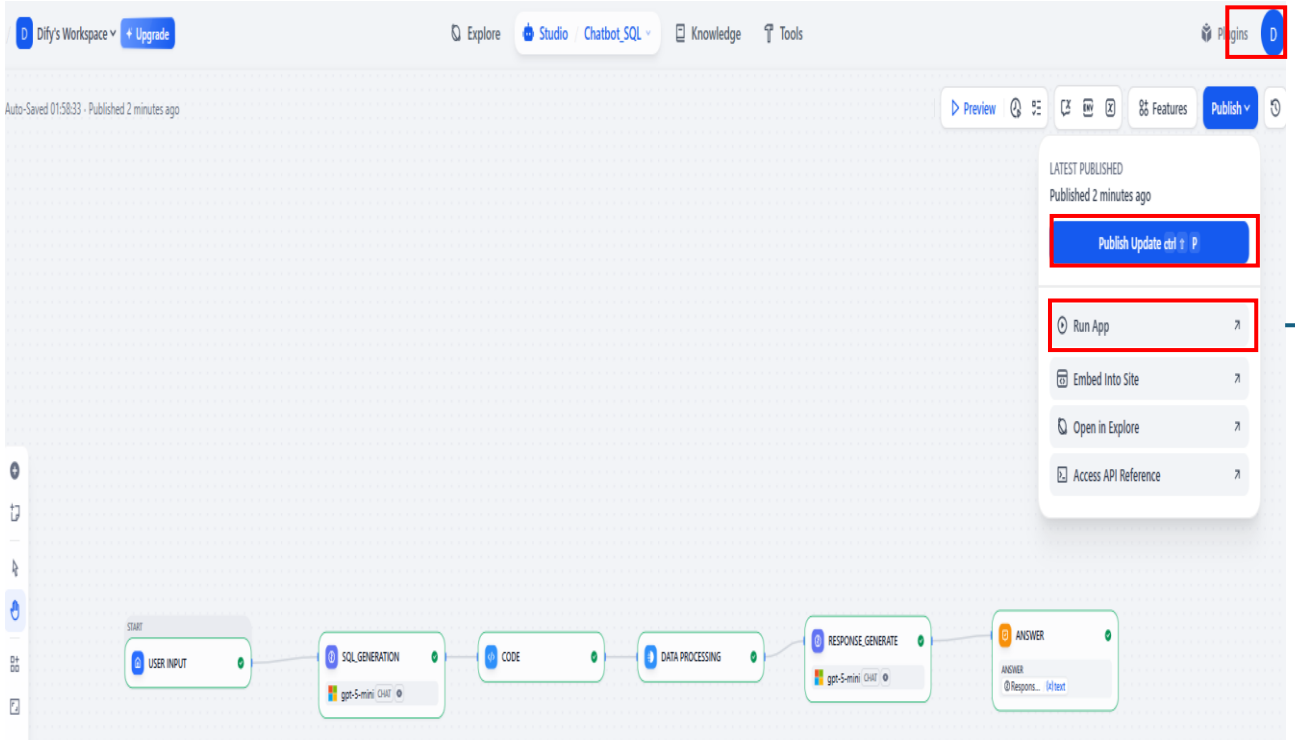
Prompt Here

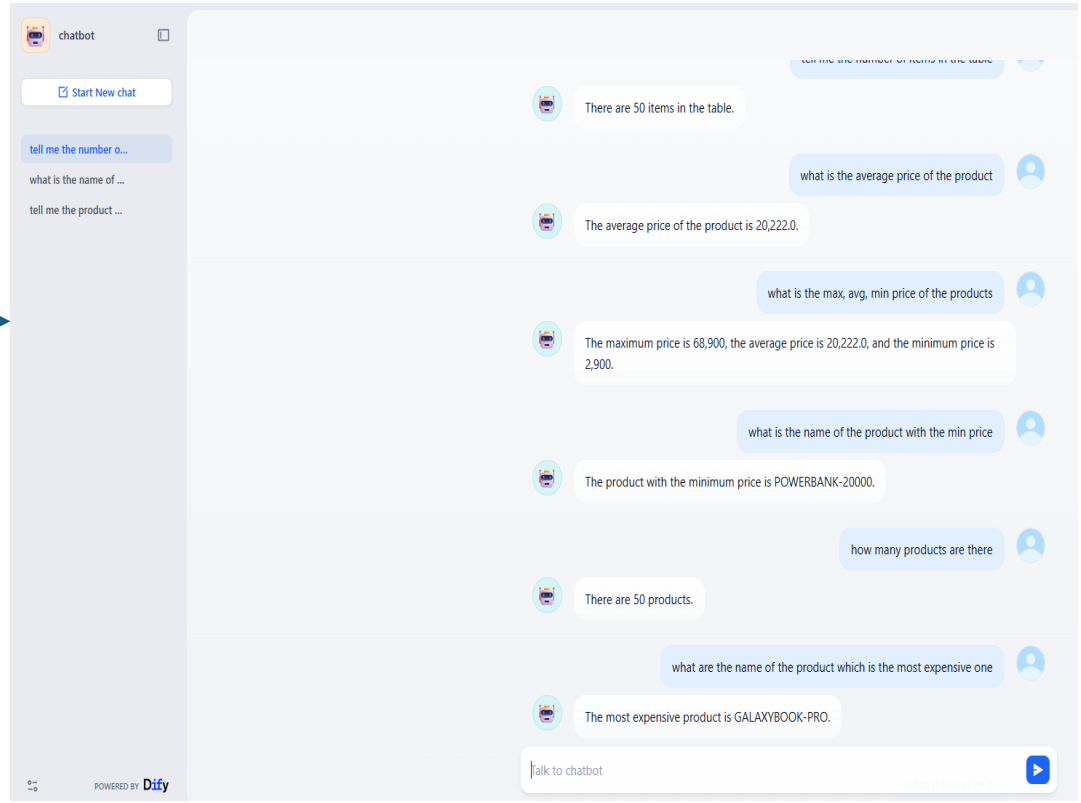
Answer Node


START
USER INPUT
SQL_GENERATION
gpt-5-mini CHAT
CODE
DATA PROCESSING
RESPONSE_GENERATE
gpt-5-mini CHAT
ANSWER
ANSWER
Respons... {x}text


Answer
Add description...
SETTINGS
LAST RUN
ANSWER
24
Respons... {x}text
NEXT STEP
Add the next step in this workflow
SELECT NEXT STEP

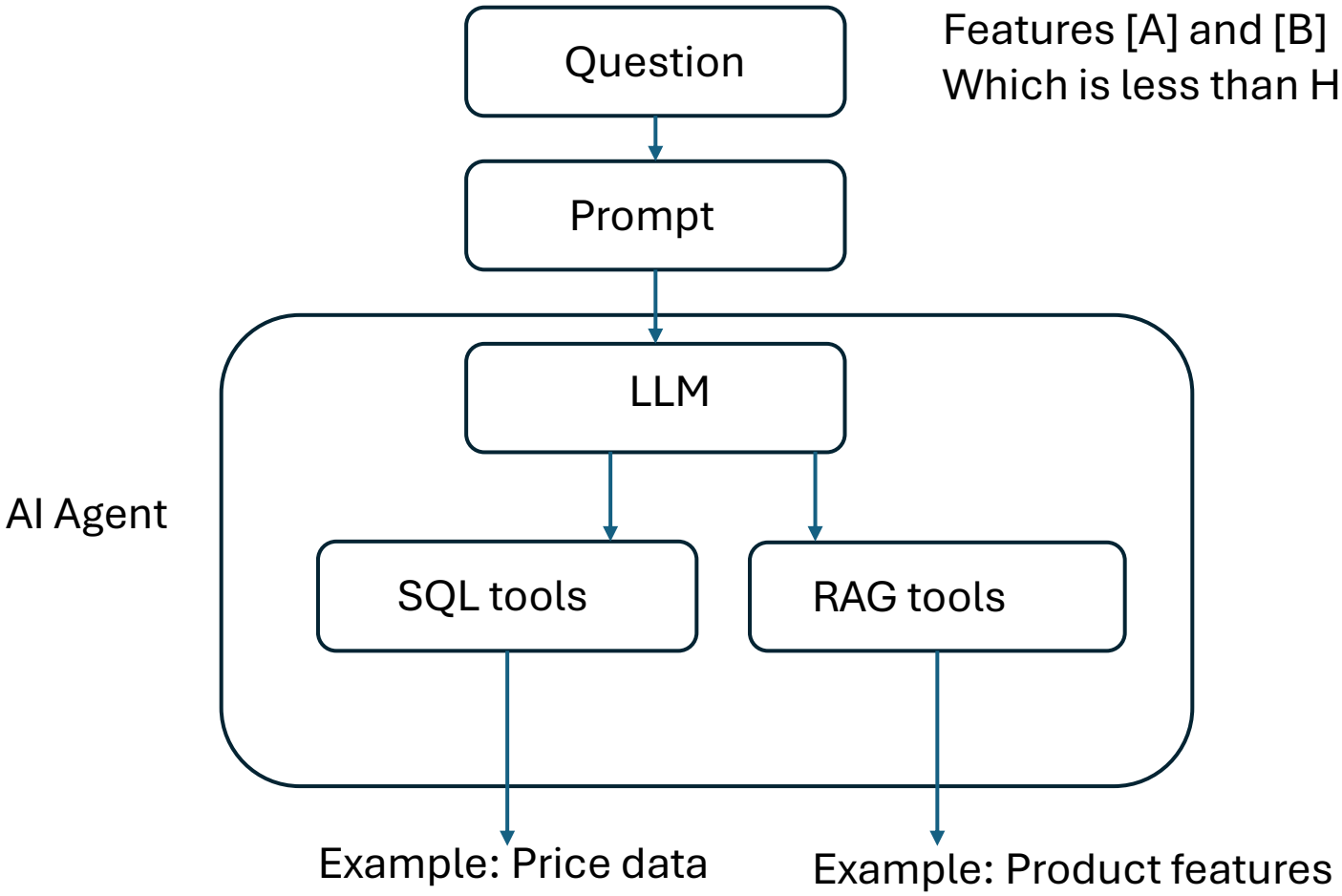
Publish the Chatbot





Multi-Hop Data/Knowledge Retrieval with AI Agent Approach

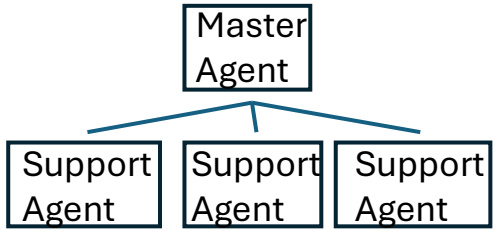
Example: Do you have any [Product] with Features [A] and [B]
Which is less than HKD 9000 ?



Design Patterns of AI Agent

Design Pattern of AI Agent

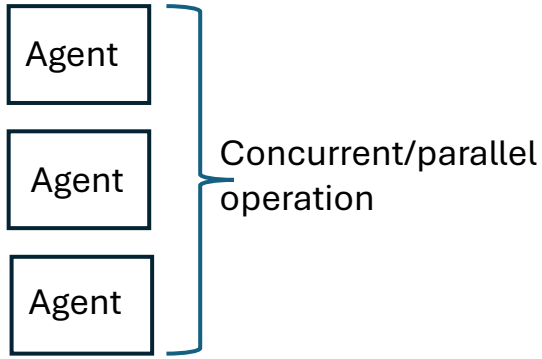
- Manager-Worker Architecture



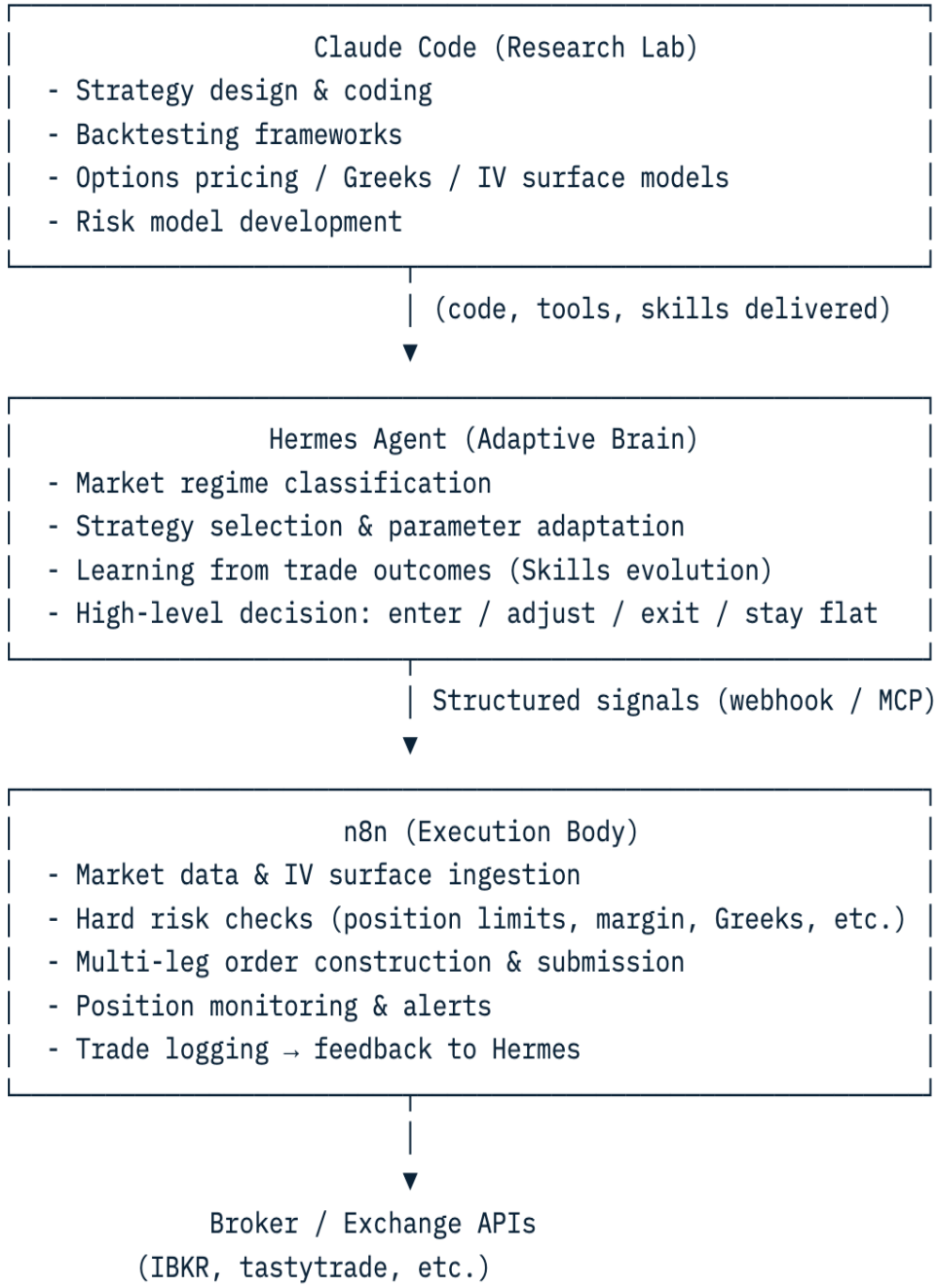
- Sequence mode



- Concurrent/parallel mode



Example of Multi-Agent deployment for Algo Trading

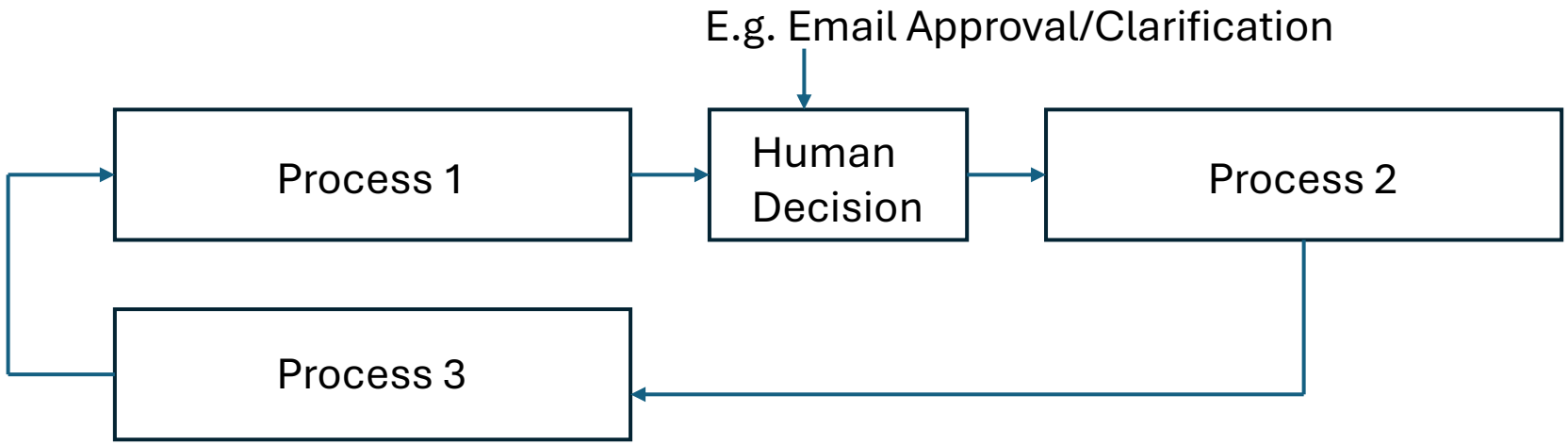


Human-in-the-loop

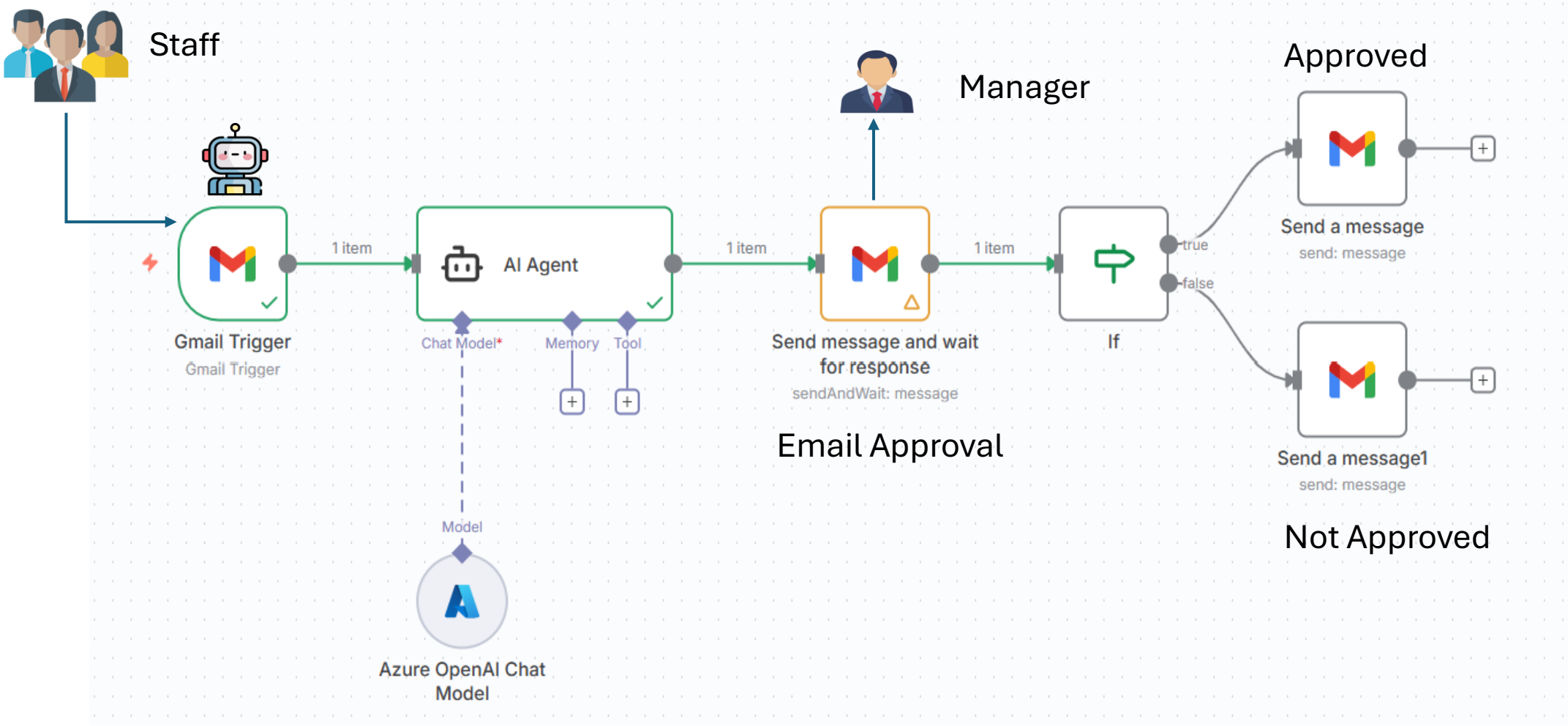
Human-in-the-loop(HITL)

The practice of incorporating human oversight into AI-driven agentic workflows.

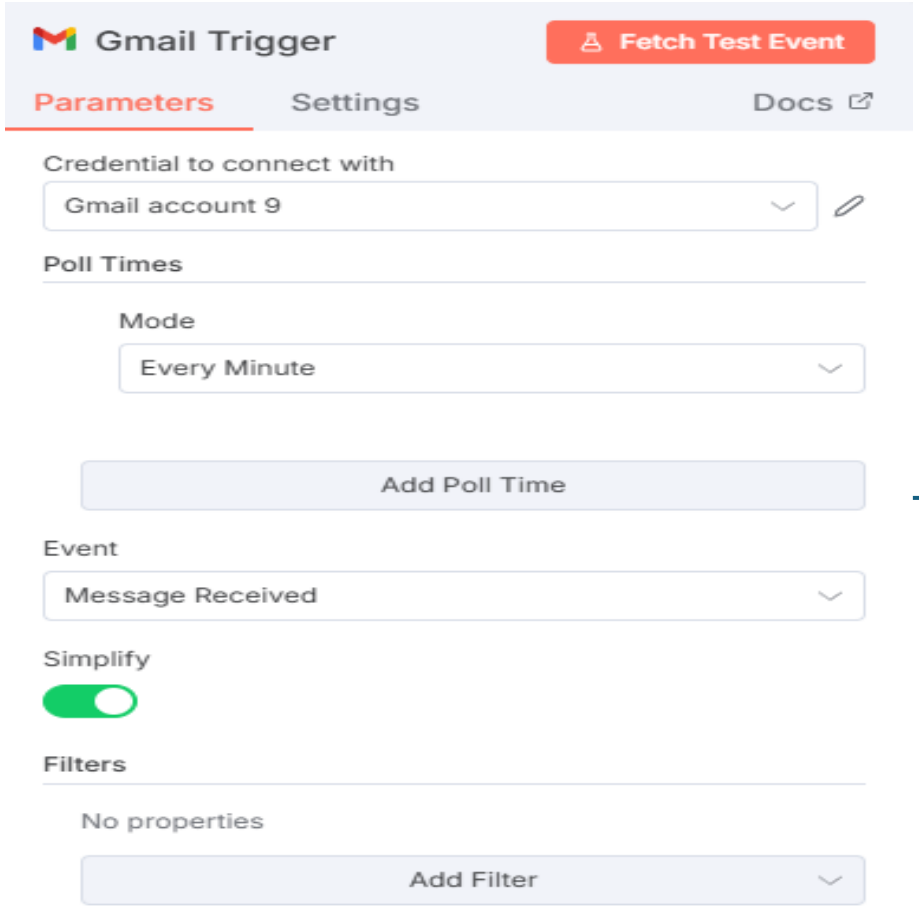
- HITL requires the AI to pause at critical points for human input, approval, or correction.
- This integration **combines AI's efficiency and scalability with human strengths in judgment, empathy, and ethical reasoning**.
- Primary goals of HITL:
  - Mitigate risks such as errors, biases, and unintended consequences
  - Ensure safety, accuracy, and regulatory compliance, especially in high-stakes scenarios

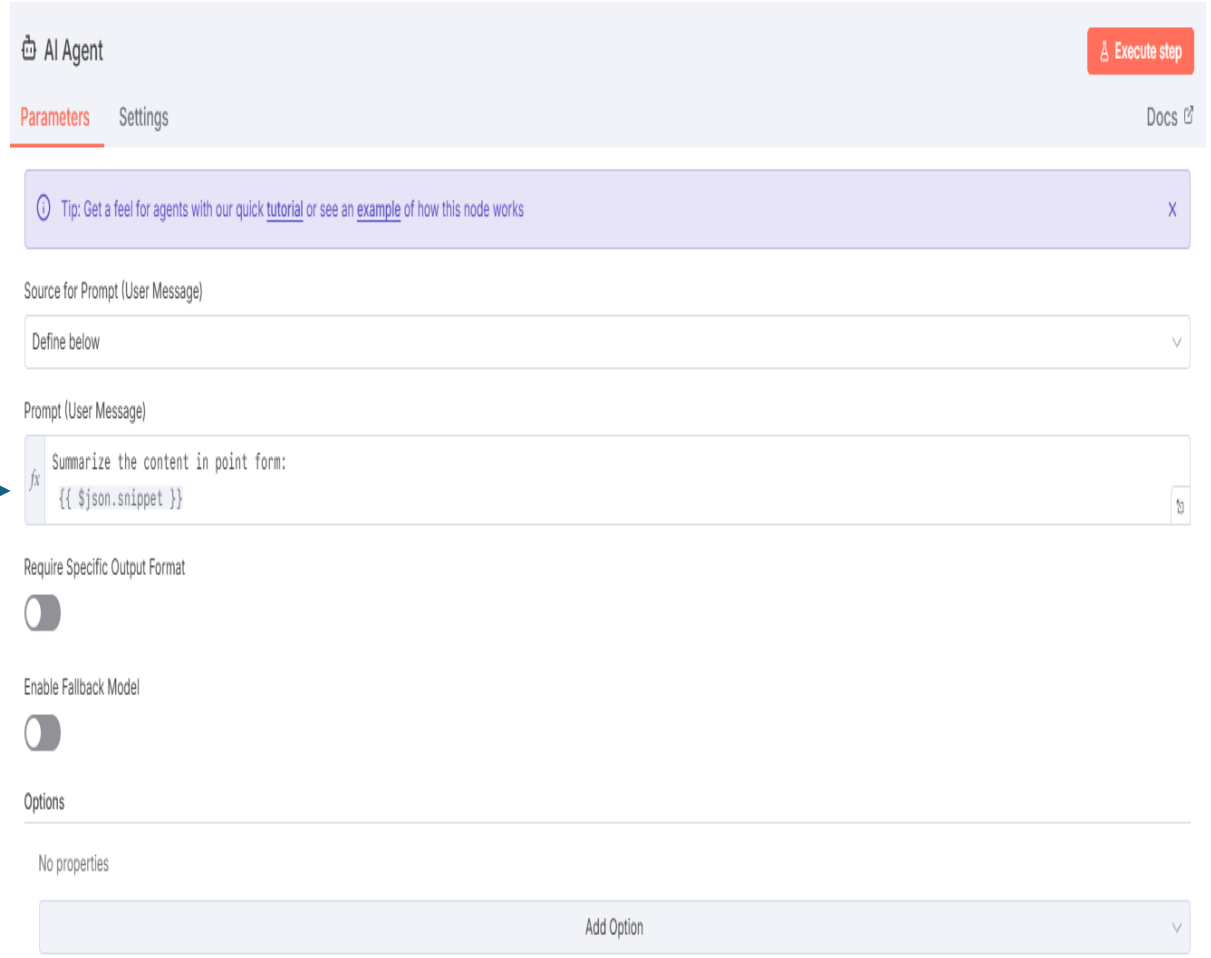


Human-in-the-loop Support

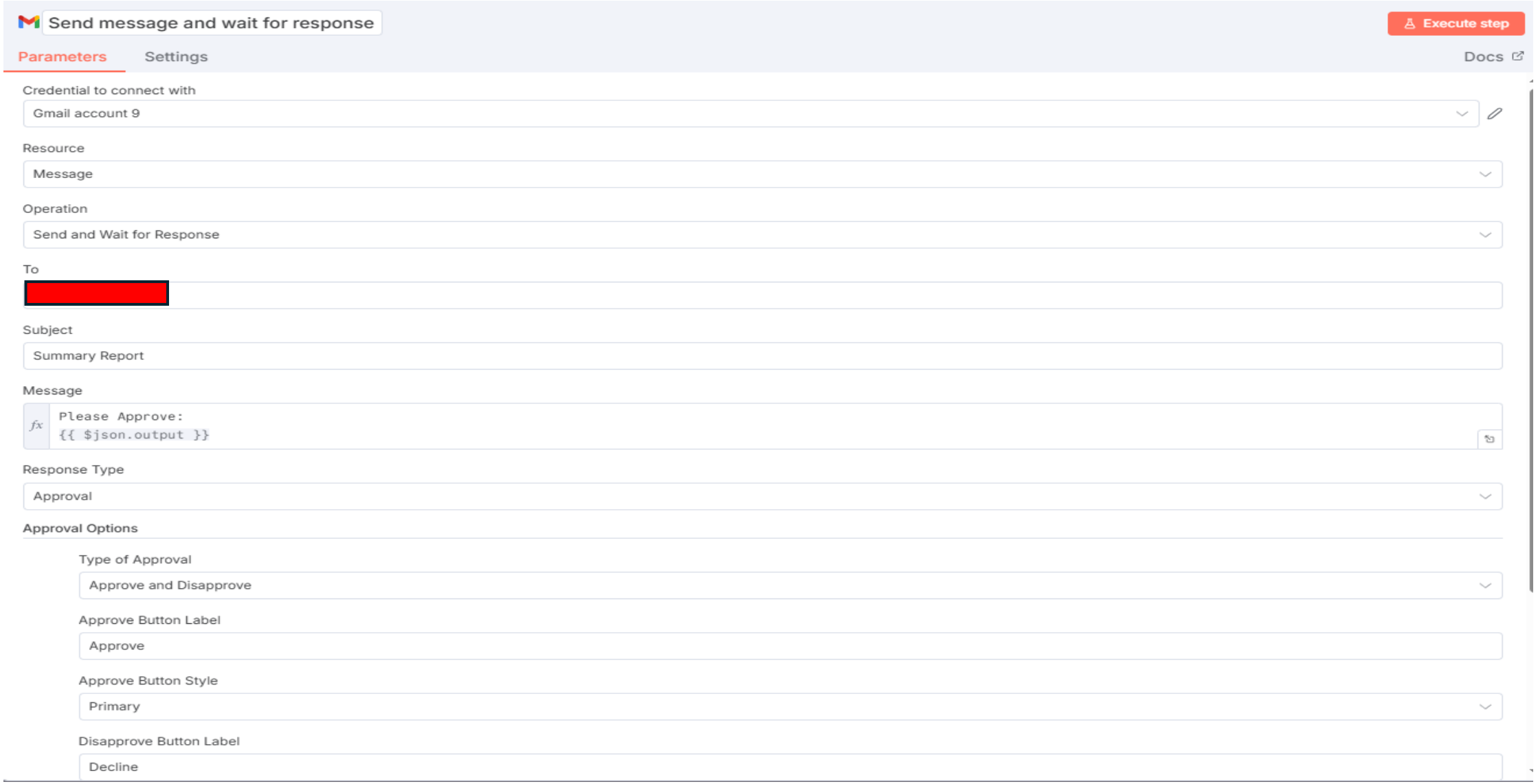


Sample Workflow-1/3





Sample Workflow-2/3



Sample Workflow-3/3

